Deep Learning Framework for Automated Plant Disease Detection Using Leaf Images

A Mini Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Or

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

2420030377 – V Sudhir Naik

2420080030 –G Akshith Reddy

2420080068 – K Harish Reddy

2420090053 –K Yaswanth Reddy

Under the Esteemed Guidance of

Dr. K Swanthana

Assistant Professor, Applied Machine Learning for Text Analytics



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Or

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DECLARATION

The mini-project report entitled "**Deep Learning Framework for Automated Plant Disease Detection Using Leaf Images**" is a record of bona fide work carried out by V Sudhir Naik (2420030377), G Akshith Reddy (2420080030), K Harish Reddy (2420080068), K Yahwanth Reddy (2420090053) in partial fulfillment of the requirements for the award of the B.Tech degree at K L (Deemed to be) University. The work presented in this report is original to the best of our knowledge, and all datasets, tools, code, and external sources used have been appropriately acknowledged.

2420030377 – V Sudhir Naik

2420080030 –G Akshith Reddy

2420080068 – K Harish Reddy

2420090053 –K Yaswanth Reddy

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Or

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CERTIFICATE

This is to certify that the mini-project report entitled "**Deep Learning Framework for Automated Plant Disease Detection Using Leaf Images**" is a record of bona fide work carried out by V Sudhir Naik (2420030377), G Akshith Reddy (2420080030), K Harish Reddy (2420080068), K Yahwanth Reddy (2420090053) in partial fulfillment of the requirements for the award of the B.Tech degree at K L (Deemed to be) University. The work presented in this report is original to the best of our knowledge, and all datasets, tools, code, and external sources used have been appropriately acknowledged.

Signature of the Project Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who has assisted us, in one way or another, in the completion of this project.

Our greatest appreciation goes to our guide, Dr. K Swanthana, Assistant Professor of the Applied Machine Learning for Text Analytics course, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance cannot be expressed in words.

We express our gratitude to Dr. P.Venkateshwar Sir (CSE)/Dr. Kaja Shareef Sir (CSIT)/Dr.Sreenath, Head of the Department for Computer Science Engineering/ Department for Computer Science and Information Technology/ Department Of Artificial Intelligence And Data Science for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the teaching and non-teaching staff members who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

2420030377 – V Sudhir Naik

2420080030 –G Akshith Reddy

2420080068 – K Harish Reddy

2420090053 –K Yaswanth Reddy

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	9
4.	Problem Statement and Objectives	12
5.	Hardware and Software requirements	13
6.	Implementation	14
7.	Results & Discussion	19

8.	Conclusion & Future Scope	23
9.	References	24

1. ABSTRACT

Agriculture plays a significant role in economic development and food security. However, plant diseases remain one of the major challenges faced by farmers, as they can reduce crop quality, productivity, and overall agricultural output. Traditional disease identification methods often depend on manual inspection and expert knowledge, which may be time-consuming and difficult to access in certain agricultural regions.

This project presents **PlantCare AI**, a deep learning-based framework designed to automatically identify plant diseases using leaf images. The proposed system uses the EfficientNetB0 convolutional neural network with transfer learning to classify plant leaf images into 15 supported disease and healthy categories involving tomato, potato, and pepper plants.

The system uses image preprocessing, dataset splitting, model training, validation, and prediction to identify the condition of a given leaf image. In addition to disease classification, the application incorporates Grad-CAM-based visual explainability to highlight the image regions that influence the model's prediction.

The framework also provides general disease-prevention and treatment guidance, an experimental severity-estimation module, and an interactive Streamlit-based user interface. The application allows users to upload leaf images, view predicted disease categories, examine prediction confidence, and access relevant plant-care recommendations.

The project combines deep learning, explainable artificial intelligence, and web-based deployment to demonstrate how automated image analysis can support preliminary plant disease screening. The system is intended as a decision-support tool and does not replace professional agricultural diagnosis.

Keywords: Plant Disease Detection, Deep Learning, EfficientNetB0, Transfer Learning, CNN, Grad-CAM, Explainable AI, Streamlit, Image Classification.

2 Team Name: Plant Care AI

Team Members and Responsibilities:

S.No.	Name	Roll No.	Role / responsibility
1.	V Sudhit Naik	2420030377	Conceptualization, methodology
2.	G Akshith Reddy	2420080030	Software, investigation
3.	K Harish Reddy	2420080068	Validation, original draft
4.	K Yaswanth Reddy	2420090053	

Project guide: Dr. K Swanthana, Assistant Professor, Department of Computer Science and Engineering, KLH Bowrampet Campus.

Dataset/source and project repository:

- PlantVillage Dataset – GitHub (original repository)
- PlantVillage – Hugging Face dataset instructions
- PlantVillage – TensorFlow Datasets documentation
- PlantVillage – Dataset information

1 INTRODUCTION

1.1 Background

Agriculture is an essential sector that contributes significantly to the economy and provides food resources for the growing population. Healthy crops are necessary for maintaining agricultural productivity. However, plant diseases caused by bacteria, fungi, viruses, and pests can seriously affect plant growth and crop production.

Early identification of plant diseases is important because delayed diagnosis can allow infections to spread across agricultural fields. Farmers traditionally identify plant diseases through visual inspection of leaves, stems, and other plant parts. Although this method is commonly used, it may require considerable experience and can be affected by environmental conditions, disease similarities, and human error.

Recent developments in Artificial Intelligence and Deep Learning have introduced new possibilities for automated plant disease identification. Convolutional Neural Networks (CNNs) can learn visual patterns from images and identify different categories without requiring manually designed image features.

PlantCare AI uses a deep learning approach to analyze plant leaf images and predict the corresponding disease category. The framework is designed to provide an accessible interface for preliminary plant disease screening and educational purposes.

1.2 Motivation

The motivation behind this project is to develop an intelligent and accessible system that can assist in the early identification of common plant diseases.

The major motivations include:

- Reducing dependence on manual visual inspection for preliminary disease screening.
- Applying deep learning techniques to agricultural image classification.
- Providing a simple interface for uploading and analyzing leaf images.
- Improving model transparency through visual explanation techniques.
- Providing general prevention and plant-care information.
- Demonstrating the practical application of Artificial Intelligence in agriculture.

1.3 Project Overview

PlantCare AI is an image-based plant disease classification system developed using Python, TensorFlow, Keras, EfficientNetB0, and Streamlit.

The system accepts a plant leaf image from the user. The image is processed into the required input format and passed to the trained deep learning model. The model generates class probabilities and identifies the most likely supported category.

The application then displays the predicted category and its confidence score. When enabled, Grad-CAM generates a heatmap that highlights image regions influencing the model's prediction.

The system also provides general plant-care recommendations and an experimental severity-estimation feature.

[INSERT IMAGE 1: PlantCare AI application home page or main dashboard]

Figure 1.1: Overview of the PlantCare AI application.

1.4 Scope of the Project

The scope of this project includes:

1. Classification of 15 supported plant disease and healthy categories.
2. Analysis of tomato, potato, and pepper leaf images.
3. Image preprocessing and normalization.
4. Transfer learning using EfficientNetB0.
5. Prediction confidence visualization.
6. Grad-CAM-based explainability.
7. General disease-prevention and treatment guidance.
8. Experimental disease-severity estimation.
9. Streamlit-based interactive application.
10. Generation of downloadable prediction information.

The current implementation focuses on image classification using the supported dataset categories. It is not intended to diagnose every possible plant disease or provide verified pesticide prescriptions.

1. Literature review

The literature review primarily focuses on transfer learning-based approaches for leaf disease detection. Comprehensive review work on plant leaf disease detection using deep learning techniques can be found in [2]. Sethy et al. [3] proposed a deep learning-based feature extraction method using an SVM as the classifier. They achieved an ac-curacy of 98.38% using the ResNet50 architecture coupled with an SVM classifier. B. Jangid [4] proposed a transfer learning-based approach for leaf disease classification, along with a Flask web-based application for making predictions. Simadhri et al. [5] applied different transfer learning approaches for 10-class leaf disease detection and identified InceptionV3 as the best performer with an accuracy of 99.64%. Hasan et al. [6] proposed a novel, comparatively smaller CNN model for the prediction of rice leaf diseases, achieving promising results. Latif et al. [7] proposed a modified VGG-19 model for accurate classification of six types of rice plant diseases. They achieved a highest average accuracy of 96.08 Andrew et al. [8] focused their work on fine-tuning hyperparameters for CNN-based pre-trained models for 38-class disease detection, where the DenseNet model outperformed others with an ac-curacy of 99.8%. Mohanty et al. [9] applied GoogleNet to the publicly available PlantVillage dataset and achieved an accuracy of 99.35% Sunil et al. [10] applied efficientNetV2 on cardamom leaf and achieved an accuracy of 98.26% Berfenati et al. [11] focused their

research primarily on unsupervised deep learning techniques for detecting pow-dery mildew in cucumber plants. Their work utilized unsupervised methods to circumvent the need for extensively labeled large datasets, which are often difficult to acquire in agricultural settings. Albattah et al. [12] proposed an improved EfficientNetV2-B4 architecture with an additional dense layer and achieved average precision, recall, and accuracy values of 99.63%, 99.93%, and 99.99%, respectively. Yuan-boye et al. [13] aimed to solve the problems of insufficient labeled data samples and complex backgrounds using an attention-improved EfficientNetV2 model for cassava leaf disease recognition. The authors achieved an recognition accuracy of 99.56%. Sujatha et al. [14] con-ducted a performance comparison of various machine learning and deep learning approaches. The classification accuracy of DL methods was found to be significantly higher than that of ML methods. The authors achieved accuracies of 87% and 89.5% using SVM and VGG-16, respectively. Guan et al. [15] introduced the Dise-Efficient model, a lightweight architecture based on EfficientNetV2, to enhance the efficiency of plant disease and pest identification using a dynamic learning rate decay strategy. For the PlantVillage dataset, Dise-Efficient achieved an impressive accuracy of 99.80%.

Jhundechen et al. [16] approached the automatic identification of plant diseases such as rice and maize using a modified VGGNet model (INC-VGGN) and achieved an accuracy of 92.8%.

Paymode et al. [17] implemented VGG model for forecasting early-stage diseases of tomato and grape plant. They achieved a classification accuracy of 98.4% for grapes and 95.71% for tomatoes.

Mehdhar et al. [18] designed a novel hybrid method by combin-ing transfer learning with the Gravitational Search Algorithm (GSA) for plant disease classification. The authors utilized MobileNetV2 and ResNetV2 for feature extraction, followed by feature concatenation. These features were then processed by the GSA for optimal feature selection, with final classification performed using Multinomial Logistic Regression (MLR). Their proposed method achieved a superior accu-racy of 99.2%. Wasswa et al. [19] introduced two transfer learning models: Plant Disease Detection Network (PDDNet) with early fusion (AE) and Late Voting Ensemble (LVE) models using EfficientNetB6, NASNetMobile, and ConvNextSmall. Logistic regression was then used for performance evaluation. The proposed models achieved 96.74% and 97.79% accuracy on the AE and LVE variants, respectively. Sha-hoveisi et al. [20] evaluated four transfer learning-based models — Xception, ResNet50, EfficientNetB4, and MobileNet — for detecting rust disease in sunflower, common bean, and field pea plants. Their findings indicated that EfficientNetB4 outperformed the other archi-tectures. Using the Adam optimizer with a learning rate of 0.001, the model achieved an average accuracy of 94.29%. Atila et al. [21] proposed the use of EfficientNet architectures for plant disease classi-fication. They evaluated their approach using the PlantVillage dataset, which contains 55,448 original images and 61,486 augmented images. Their results demonstrated that the EfficientNet-B4 and EfficientNet-B5 architectures achieved the highest accuracies, reaching 99.97% and 99.91%, respectively. Aboelenin et al. [22] developed a hybrid architecture that integrates ensemble CNN models with a Vision Trans-former (ViT). This approach was designed to enhance the feature extraction capabilities of CNNs by leveraging the global context aware-ness provided by the Vision Transformer. Song et al. [23] designed a novel Dynamic Dual-Stream Fusion Network (DS-FusionNet) to ad-dress technical challenges such as small-sample learning, leaf occlusion, illumination variation, and inter-class similarities. Their architecture integrates EfficientNet-B4 with a ConvNeXt-Tiny dual-backbone, uti-lizing a bidirectional knowledge distillation strategy. This approach exhibits outstanding performance even when applied to complex, high-variability datasets. Kour et al. [24] designed a novel hybrid model for two challenges like intra class pattern variation and inter class morphological similarities in leaf diseases. This study proposes a novel Hybrid-Convolutional Support Machine (H-CSM) framework that syn-ergistically combines Convolutional Neural Networks (CNN) for hi-erarchical feature extraction, Convolutional Block Attention Module (CBAM) for spatial and channel-wise feature refinement, and Support Vector Machine (SVM) for robust classification. The model was trained and validated on a comprehensive plant disease dataset

encompassing 14 plant species and 36 disease categories. Experimental results demon-strate that H-CSM achieves 98.72% classification accuracy, outper-forming existing state-of-the-art methods.conventional centralized deep learning approaches raise concerns regarding data privacy and com-putational resource requirements, particularly in resource-constrained agricultural settings. Aggarwal et al. [25] addresses these challenges by proposing a federated learning (FL) framework integrated with Inter-net of Agriculture Things (IoAT) infrastructure for privacy-preserving, resource-efficient rice leaf disease classification. They investigated two distinct FL strategies: federated transfer learning and federated feature extraction, evaluating their performance across classification metrics (accuracy, precision, recall, AUC) and computational efficiency metrics (GPU memory consumption, CPU/GPU utilization, virtual memory). Using a dataset of 5932 labeled images spanning four

InceptionResNetV2 achieved the highest validation accuracy of 88%. Experimental results demonstrate that transfer learning models signif-icantly outperform conventional machine learning classifiers for rice disease classification tasks. Based on the literature survey, it is evident that Deep Learning (DL) approaches yield remarkable results compared to traditional Machine Learning methods. However, DL architectures typically require massive amounts of labeled data and high compu-tational resources, often leading to overfitting on training data and poor generalization on test datasets [27]. This pose a significant chal-lenge in resource-constrained environments. Consequently, Transfer Learning serves as an optimal solution, allowing pre-trained models to be effectively utilized even on smaller datasets. Through fine-tuning, the performance can be further optimized, offering superior accuracy, reduced computational overhead, and faster convergence [28]. Toward this goal, the primary contributions of this work are highlighted as follows:

1. An optimized resource allocation method of plant disease detec-tion technique is proposed for sustainable agriculture.
2. Extensive statistical-based study on transfer learning approaches and develop an EfficientNetV2-based light-weight model for plant disease detection through fine-tuning.
3. An extensive study of the model was conducted using the PlantVillage dataset, comprising 20,639 samples across 15 image classes, along with a real-time drone dataset of 4432 images.
4. Comparative performance analysis of the proposed model, Effi-cientNet variants, and other transfer learning approaches using different statistical analyses.
5. Model explainability is achieved through Grad-CAM visualiza-tion of the diseased leaf images. This technique generates a heatmap that highlights the specific regions of interest (ROIs) within the leaf, allowing for a better interpretation of the fea-tures the model uses to identify the disease.

The primary objective of our work is to enhance performance by systematically comparing state-of-the-art models in order to identify the most effective one. The remainder of this paper is organized as follows: Section 2 details the Materials and Methods employed in the proposed work, including dataset preparation and architecture design. Section 3 provides a comprehensive Results and Discussion analysis, highlighting the performance of the fine-tuned model. Finally, the Conclusion and Future Work are presented in Section 4. As highlighted in prior research, there has been a notable increase in the application of deep learning architectures for the diagnosis of plant leaf diseases. Despite this progress, significant gaps remain, particularly concerning the exploration and implementation of novel deep learning models. There is an evident need for more efficient architectures that not only reduce the number of trainable parameters and accelerate training times but also

maintain high diagnostic accuracy. Addressing these challenges is crucial for the development of scalable, real-time disease detection systems in agricultural settings.

This study proposes the EfficientNet [28] based deep learning ar-chitecture for the classification of plant diseases. EfficientNet addresses the ‘‘gaps’’ mentioned through a technique called Compound Scaling. Unlike traditional models that scale only depth (ResNet) or width (WideResNet), EfficientNet scales all three dimensions — depth, width, and resolution — using a fixed set of scaling coefficients. This method-ology ensures that the model remains ‘‘lightweight’’ while capturing the complex, fine-grained features necessary for identifying subtle leaf lesions. While EfficientNet provides a robust feature extraction back-bone, its standard implementation is often fully frozen or fully trained. This study implements a selective fine-tuning strategy, unfreezing the last two blocks along with the classification head of the architecture to allow the model to specialize its high-level representation for agricul-tural textures while retaining the generalized spatial patterns learned from the pre-trained dataset.

1.

Materials and methods

In this section, we discuss about the dataset used for the experiment and the architecture of the proposed model. The baseline model for the classification task is the EfficientNet architecture.

1. *Description of the dataset*

For performance evaluation, two widely adopted datasets were utilized. The first dataset, known as the PlantVillage dataset [9], is publicly available at www.kaggle.com/datasets/emmarex/plantdisease and has been extensively used in plant disease detection research. The second dataset comprises of real-time drone-captured images [3]. A detailed description of both datasets is provided below.

The PlantVillage dataset utilized in this study contains a variety of diseased and healthy leaf images across different species, specif-ically potato, tomato, and bell pepper. This subset comprises of 15 classes with a total of 20,639 images. In addition to the pathologi-cal samples, the dataset includes healthy leaf images for each plant species to serve as a baseline. Fig. 1 provides a comprehensive break-down of the dataset, illustrating the specific disease categories and the corresponding image counts.

To evaluate the model’s robustness in field conditions, a second dataset consisting of real-time drone-captured imagery [3] was em-ployed. This dataset contains 4432 images of rice plants affected by three major conditions: Bacterial Blight, Leaf Blast, and Brown Spot.

To enhance model robustness and prevent overfitting, a rigor-ous data augmentation pipeline was implemented during the training phase. Input images were resized to 224 × 224 pixels and subjected

to stochastic transformations, including random horizontal flipping, rotations up to 15◦, and color jittering (adjusting brightness, contrast, and hue by ±20%). Furthermore, a random resized cropping strategy

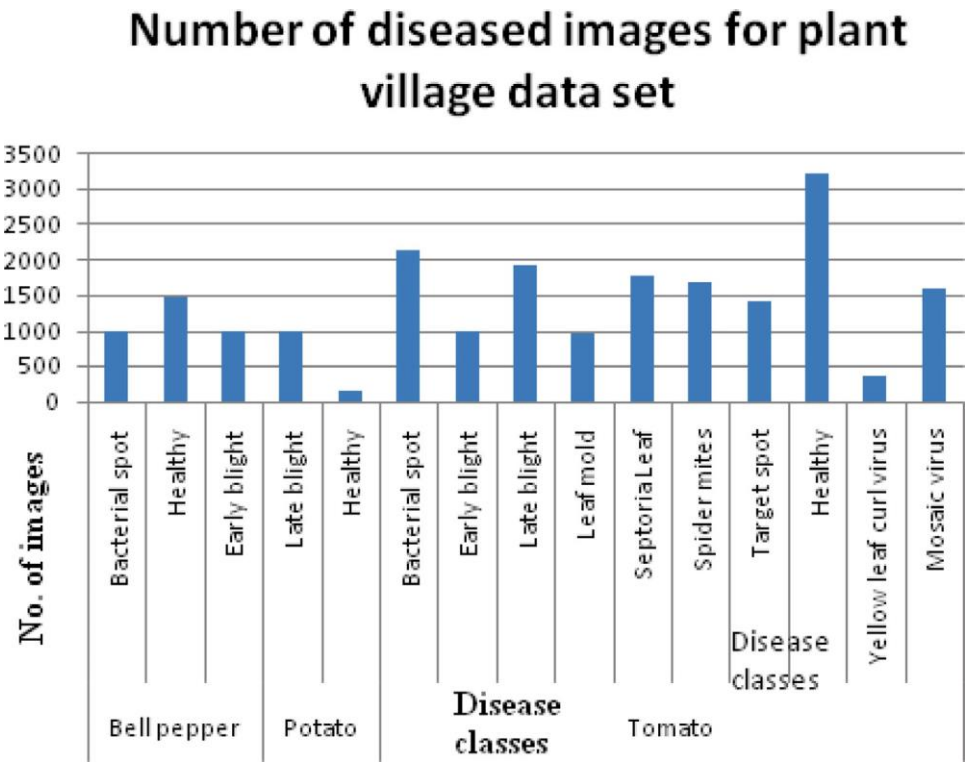
(scale 0.8 to 1.0) was applied to simulate various zoom levels. Finally, the data was normalized using ImageNet-derived mean and standard deviation values ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$) to ensure compatibility with the pre-trained EfficientNetV2-B1 weights.

The drone captured dataset contains three types of diseased im-age for rice leaf namely bacterial blight, blast and brown spot. Full description is provided in Fig. 2.

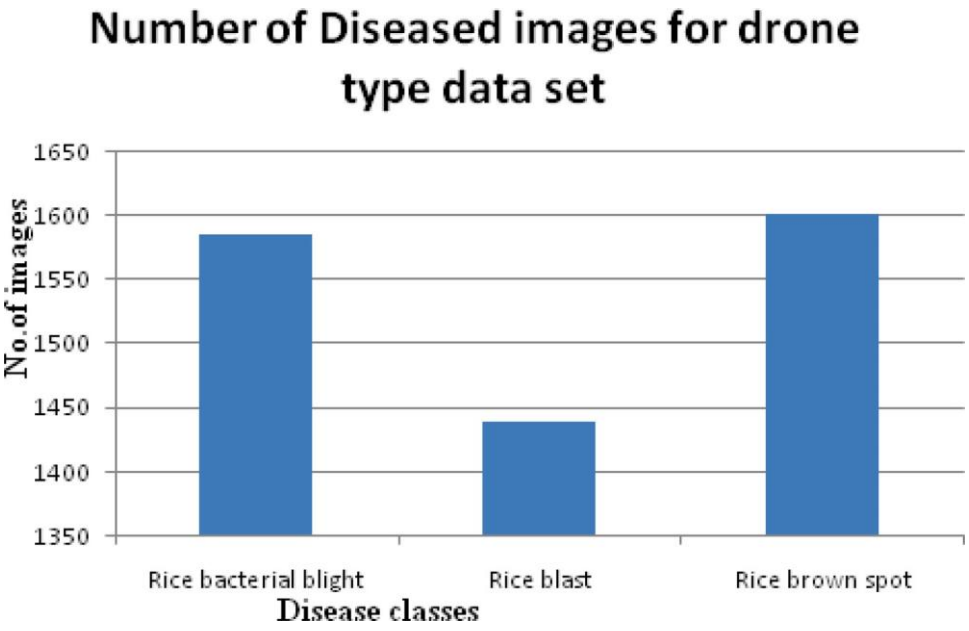
Sample images of plant village dataset and drone captured dataset are given in Figs. 3 and 4.

1. *Architecture*

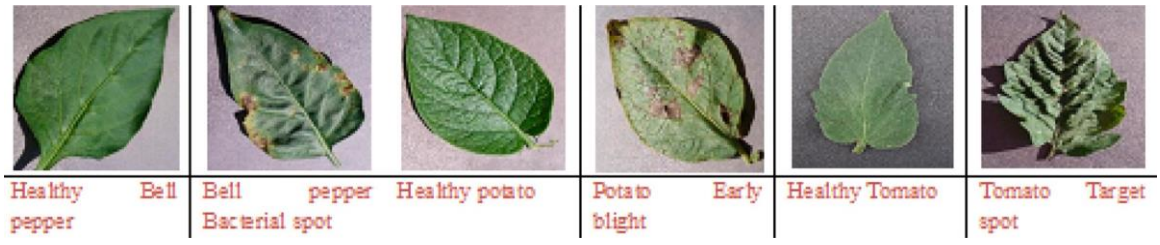
The flowchart of the proposed framework is shown in Fig. 5. A fine-tuned EfficientNet framework is proposed for leaf disease detection. For this disease detection task, a pre-trained EfficientNet model was deployed that fits the proposed work and demonstrates strong clas-sification performance [28]. By freezing the initial layers, the model accelerates training and improves feature extraction efficiency by re-taining generalized spatial patterns. Subsequently, the final layers are unfrozen and new trainable layers are appended to adapt the learned representations to the specific characteristics of the leaf disease dataset. The proposed architecture employs EfficientNet as the backbone net-work, which natively utilizes Squeeze-and-Excitation (SE) modules to perform channel-wise feature recalibration. This is further enhanced with attention mechanisms to ensure robust feature extraction and accurate disease localization. The Squeeze-and-Excitation (SE) mod-ule performs adaptive channel-wise feature recalibration, allowing the network to emphasize informative feature maps while suppressing less relevant ones. Concurrently, the attention mechanism focuses the model's focus on disease-relevant spatial regions, such as necrotic spots or chlorotic halos. Following feature extraction, Global Average Pooling (GAP) aggregates the spatial dimensions into a compact feature vector. This representation is then processed through fully connected



/ig. 1. Description for plant village dataset.



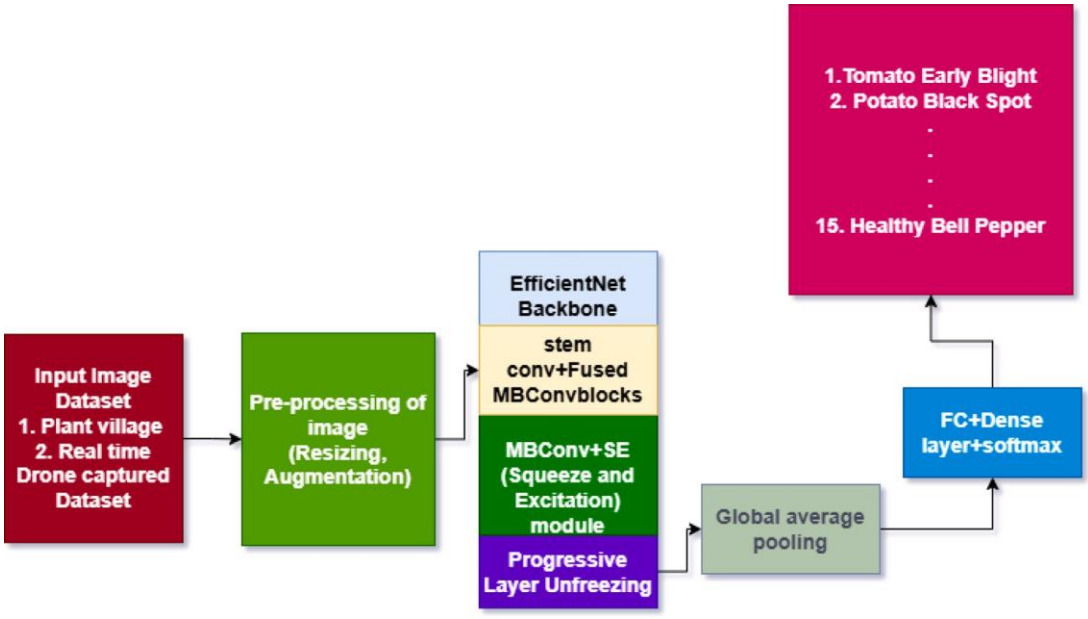
/ig. 2. Description for drone image dataset.



/ig. 3. Sample images from plant village dataset.



/ig. 4. Sample images from drone type dataset.



/ig. 5. Dataflow diagram of leaf disease detection model using EfficientNet.

(dense) layers with a Softmax activation function to produce the fi-nal probability distribution for multi-class disease classification. This architecture achieves high accuracy while maintaining computational efficiency through the use of mobile inverted bottleneck convolutions and attention-based feature refinement. Here the pre-trained model is loaded without final output layer. After adding new layers those needs to be trained for the proposed design. The base model output is different because of difference in number of classes and features. Performance of any model may improve by fine tuning the parameters which is done by un-freezing the whole model or part of it. Then the entire new model is trained at a low learning rate which prevents over-fitting.

1. *EfficientNet architecture*

EfficientNet is a CNN architecture and uniform scaling method for all dimensions using a compound co-efficient which provides better accuracy. EfficientNet architecture comes out with seven models of different dimensions. In compound scaling method width, depth and resolution scaled to a constant ratio so that dimensions are properly balanced [28].

$$d = \alpha^{\phi}$$

$$w = \beta^{\phi}$$

$$r = \Gamma^{\phi}$$

such that $\alpha * \beta^2 * \gamma^2 \approx 2$, $\alpha \geq 1$, $\beta \geq 1$, $\gamma \geq 1$. Where α, β, γ are depth scaling factor, width scaling factor and resolution scaling factor, determined using grid search algorithm. ϕ is compound scaling factor,a user defined FLOPS (Floating Point Operations per sec) The constraint

$\alpha * \beta^2 * \gamma^2 \approx 2$ ensures that the total computational cost approximately doubles when ϕ increases by 1.

$$\text{FLOPs} \propto d \cdot w^2 \cdot r^2 \quad (1)$$

Instead of redesigning the network architecture itself (the layer struc-ture Fi), model scaling focuses on expanding three dimensions of an existing baseline network:

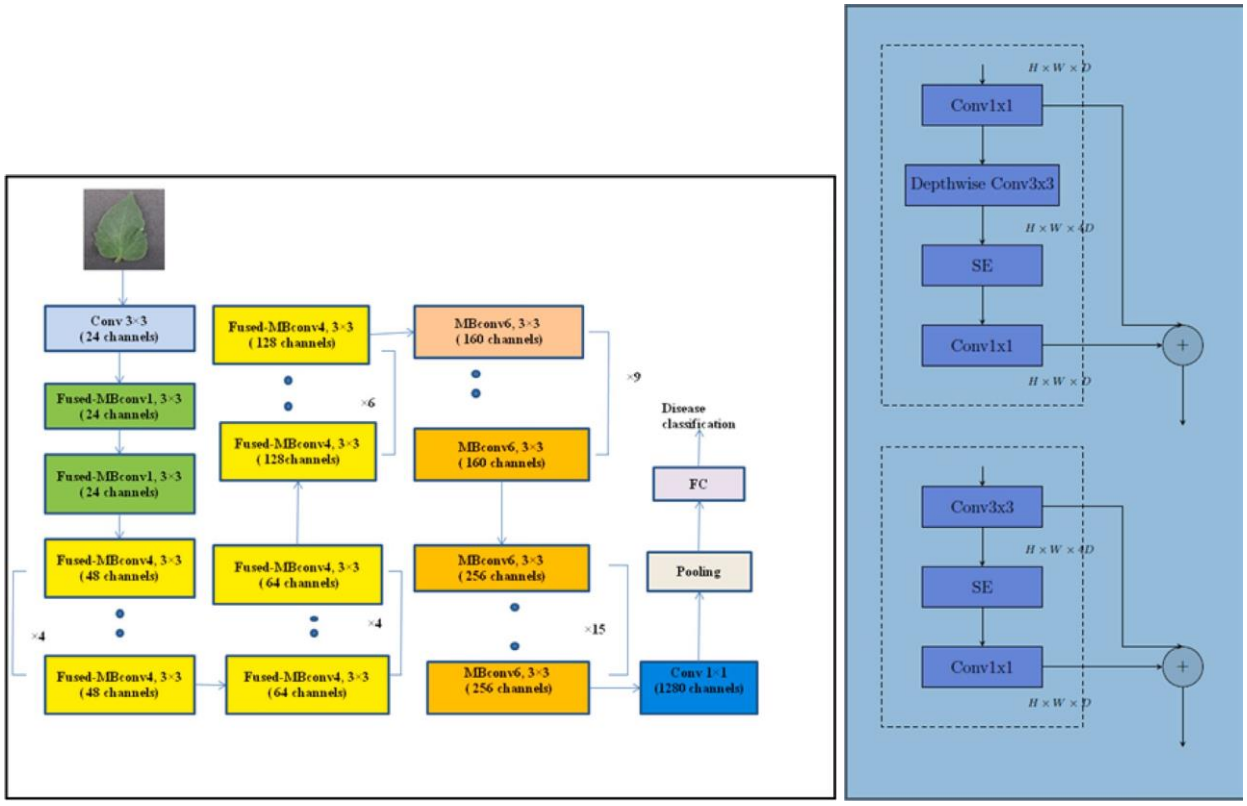
Depth (Li): Number of layers

Width (Ci): Number of channels per layer Resolution (Hi, Wi): Input image size

To optimize the performance two main constraints are there like memory uses and computational complexity. EfficientNetV2 builds on compound scaling but optimizes FLOPs through fused convolutions (e.g., MBConv with depthwise-separable convs), progressive learning, and training-aware scaling to reduce latency and parameters while maintaining accuracy. The equation $\mathcal{M}(d, w, r)$ basically defines new Network ($\mathcal{M}$) is just an old network with its layers repeated d times, its channels multiplied by w, and its image size multiplied by r

$$FLOPs = 2 \cdot H \cdot W \cdot (Cin \cdot K^2 + 1) \cdot Cout \quad (2)$$

Where: H, W: Height and width of the output feature map $Cin, Cout$: Number of input and output channels. K: Kernel size (e.g. 3 for a 3 × 3



(a) EfficientNetV2-S architecture (b) MBConv and Fused MBConv Block archi-tectures

/ig. 6. EfficientNetV2-S architectures.

conv). The factor of 2 accounts for the multiply-accumulate (MAC) operation (one multiplication and one addition). For the entire model total flops are counted by following formulae.

Table 1

EfficientNetV2-S architecture details.

Stage Operator Stride Channels Layers

i

EfficientNetV1-B0 consists of nine stages. Excluding the initial 3 × 3 convolution and the final 1 × 1 convolutional layer (which includes pooling and a fully connected head), the core architecture relies on Mobile Inverted Bottleneck Convolution (MBConv) building blocks. The MBConv layer employs depthwise separable convolutions, which sig-nificantly reduces the parameter count and computational complexity compared to standard convolutional layers while maintaining robust feature representation.

3.3.2.

EfficientNetV2: The EfficientNetV2 architecture optimizes the orig-inal MBConv structure by introducing Fused-MBConv blocks in the earlier stages of the network. Architecture of MbConv and Fused MB-Conv is provided in Fig. 6.(b). Unlike the standard MBConv, which uses a 1 × 1 expansion convolution followed by a 3 × 3 depthwise convo-lution, the Fused-MBConv replaces these with a single, regular 3 × 3 convolution. This modification significantly improves training speed and memory throughput on modern accelerators. While V1 heavily utilized 5 × 5 kernels, EfficientNetV2 favors 3 × 3 kernels and removes the final stride stage found in V1 to maintain higher resolution feature maps. The detailed configuration of the proposed EfficientNetV2 model is summarized in Table 1.

Model Representations of EfficientNetV2: EfficientNetV2 is an ad-vanced convolutional neural network architecture designed to enhance

5	MBConv4,K3*3,SE 0.25	2	160	9
6	MBConv4,K3*3,SE 0.25	1	256	15
7	Conv1*1 , Polling , FC	–	1280	1

both training speed and parameter efficiency compared to its predeces-sors. It builds upon the original EfficientNet model by incorporating several key innovations. The EfficientNetV2 base architecture is the V2-S model (Fig. 6(a)), which can be scaled to EfficientNetV2-M and EfficientNetV2-L variants. An improved compound scaling method was employed with architectural modifications to optimize both model size and inference speed. To optimize GPU memory utilization and accelerate training, the maximum input image resolution is restricted to 480 × 480 pixels. This constraint ensures a high training throughput while maintaining sufficient detail for identifying small-scale patholog-ical features. Furthermore, additional layers are appended at the final stage of the network to enhance the model's capacity to represent com-plex disease features. This strategy allows for a more expressive feature mapping without significantly increasing the computational overhead during real-time inference. EfficientNet incorporates compound scaling into CNN architectures, utilizing MBConv (Mobile Inverted Bottleneck Convolution) blocks as the foundational building blocks. In later con-figurations, EfficientNetV2 employs a combination of Fused-MBConv

and MBConv blocks to enhance training efficiency. Specifically, the Ef-ficientNetV2 architecture uses Fused-MBConv blocks for the first three stages and standard MBConv blocks for subsequent stages It makes sys-tem smaller and faster. The architecture of MB-conv and Fused MB-conv are provided in the figure (Fig. 6(b)). Incorporating EfficientNetV2 into applications like plant disease detection leverages these architectural advancements to achieve high accuracy and efficiency, making it a suitable choice for real-time image classification tasks where compu-tational resources may be limited. The initial regularization parameter and image size are defined by the programmer. Linear interpolation is then applied to increase the image size and adjust the regularization after a particular stage. Generally, three types of regularization are used in EfficientNet: Dropout, RandAugment, and MixUp. Multiple studies have demonstrated that EfficientNetV2-M achieves superior performance compared to EfficientNetB7 (previously the state-of-the-art in the EfficientNet family) when trained under identical progressive learning conditions [28].

Experimental set up:

Input: Download the images from requested website Configuration: Import the images

Configure training, testing and validation dataset Set the directories and resizes the images

Load the efficientNetV2 base model Freeze the top layers weights

Create the customized model with base model Compile the model:

Optimizer = ADAM

Loss = categorical cross-entropy Metrics = Accuracy

Learning Rate = 0.001 Train the model:

Epochs = 10

Algorithm 1 Proposed FT-EfficientNetV2 for Multi-Species Leaf Disease Detection 1: **Input:** Leaf image dataset $\mathcal{I} = \{i1, i2, \ldots, in\}$, Pre-trained

EfficientNetV2 parameters θ

2: **Output:** Predicted labels $\mathcal{L} = \{l1, l2, \ldots, ln\}$, Performance metrics $\mathcal{P}$ (Accuracy, F1-Score)

3: **Step 1: Data Preprocessing** each image $i \in I$

4: Resize i to 224×224 pixels

5: Apply scaling and normalization to range $[0, 1]$

6: Apply data augmentation (rotation, flipping, and zooming)

7: Step 2: Model Architecture Setup

8: Apply **Stem Layer**: 3×3 Convolution, Stride 2 each stage $s \in$

$\{1, 2, \ldots, 7\}$ $s \in \{1, 2, 3\}$

9: Integrate **/used-MBConv** Blocks (3x3 Conv)

10: Integrate **MBConv** Blocks with **Squeeze-and-Excitation (SE)**

11: Initialize weights θ from ImageNet pre-training

12: Step 3: Empirical Progressive /ine-Tuning

13: Freeze the initial backbone stages to preserve general features 14: Unfreeze the **last 8 layers** of the EfficientNetV2-B1 backbone 15: Configure customized classification head with **Dropout (0.2)** 16: **Step 4: Model Training and Inference**

17: Execute training using **Adam Optimizer** ($LR = 0.0001$) for 10 epochs each image $i \in I$

18: Generate feature vector via Global Average Pooling

19: Compute Softmax probability $P(y|i)$

20: Assign label $li = \text{argmax}(P(y|i))$

21: Compute P via 10-fold Cross-Validation and Confusion Matrix

22: **Return** L, P

This algorithm describes a fine-tuning pipeline for EfficientNetV2-B1 applied to leaf disease detection. M.features represents the feature extractor (backbone) of EfficientNetV2-B1, which outputs high-level feature maps from input leaf images.M.head represents the complete

model (EfficientNet backbone + classification head) used for training and inference.z is the predicted output of the network for a mini-batch (from the forward pass). X represents a mini-batch of input leaf images from the dataset and Y is the ground-truth labels for the current mini-batch. For leaf disease detection, a fine-tuned EfficientNetV2-B1 model leverages transfer learning by freezing the early feature extraction stages (blocks 0–6) while selectively unfreezing the final two stages

— feature blocks 7 and 8 — along with a custom classifier head. This progressive unfreezing strategy preserves the pre-trained generic visual features learned from ImageNet (such as edges, textures, and basic plant structures) in the initial MBConv and Fused-MBConv blocks, preventing catastrophic forgetting during adaptation to plant pathology datasets. The unfrozen final blocks (typically producing $7 \times 7 \times 1280$ feature maps) and classifier head (GlobalAvgPool → Dropout → Linear to disease classes). The pooled features from both blocks are then con-catenated to form a unified feature representation. Unlike traditional transfer learning approaches where the classifier head is frozen, the classifier head in this framework is kept unfrozen. This fine-tuning strategy allows the model to adapt more effectively to disease-specific characteristics present in leaf

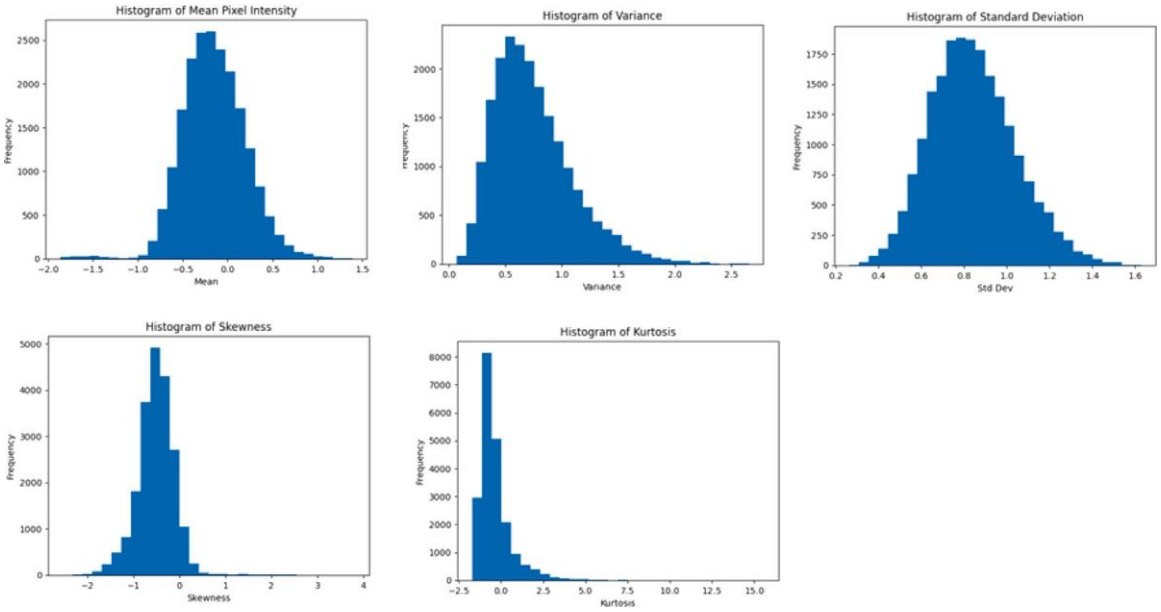
images. The combined features are passed through fully connected layers with batch normalization, activation functions, and dropout for regularization, followed by a softmax layer for multi-class disease classification. This fine-tuning strategy allows the model to adapt more effectively to disease-specific characteristics present in leaf images. The combined features are passed through fully connected layers with batch normalization, activation functions, and dropout for regularization, followed by a softmax layer for multi-class disease classification.

1. Simulation and result analysis

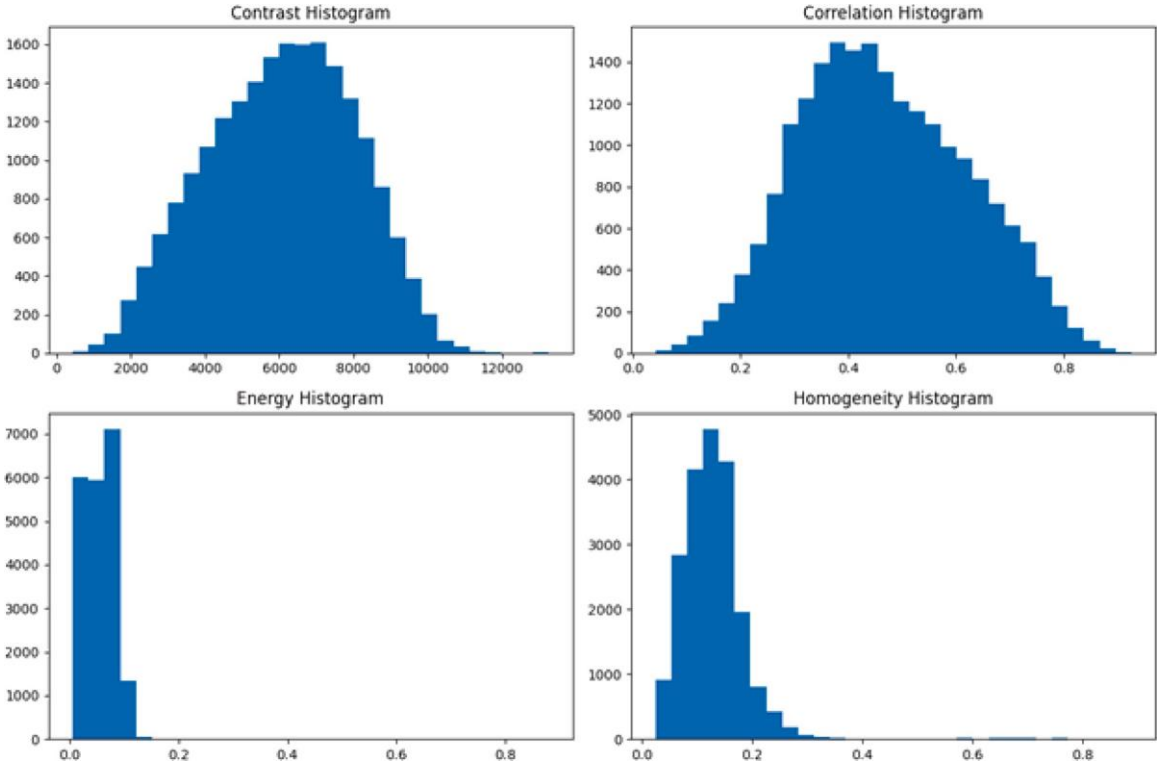
In this section, we discussed about different performance measures used for the evaluation of the algorithm followed by simulation and result analysis. For the performance evaluation, we have considered different versions of EfficientNetV2 such as EfficientNetV2-B0, Effi-cientNetV2-B1, EfficientNetV2-B2, EfficientNetV2-B3, EfficientNetV2-S, EfficientNetV2-M, EfficientNetV2-L with fine-tuning. Performance of all the models are evaluated based on average recall, average precision, average F1 score and Average accuracy. Prior to experimenting with the model's performance, we discuss the statistical behavior of our dataset in the next paragraph. Lastly we analyzed different statistical parameters for model verification.

We analyzed the first-order statistics and second-order statistics related to the features of the dataset used in Figs. 7 and 8 respectively. The global intensity characteristics of pixels are described by first-order statistics of an image dataset, such as mean, variance, skewness, and kurtosis. These statistics capture the average brightness or darkness of the images as well as the degree of dispersion or skewness of the pixel values. By simulating the interactions between pairs of adjacent pixels, second-order statistics — which are frequently generated from techniques like the Gray Level Co-occurrence Matrix (GLCM) — char-acterize texture and produce metrics like contrast, correlation, energy, and homogeneity. These statistics are significant because they produce interpretable, low-dimensional features appropriate for classification while compactly representing both local structural patterns and global illumination characteristics, allowing for robust discrimination between healthy and diseased regions in plant-leaf images.

Second-order statistics obtained from the Gray Level Co-occurrence Matrix capture the spatial dependency between neighboring pixels, whereas first-order statistics explain the distribution of pixel intensi-ties without taking spatial interactions into account. Strong intensity changes between neighboring pixels are indicated by the acquired data's high contrast value (6060.39), which is compatible with the existence of disease lesions. The low energy (0.0589) and homogeneity (0.1307) show uneven and non-uniform texture patterns typical of sick leaf surfaces, while the moderate correlation (0.466) shows partial structural reliance across pixels. First order and second order statistics parameters are provided in Tables 2 and 3.



/ig. 7. First-order feature histogram statistics for 15 class dataset.



/ig. 8. Second order feature histogram statistics for 15 class dataset.

1. *t-SNE chart(t-distributed stochastic neighbor embedding*

Fig. 9 presents the t-SNE analysis of the 15-class PlantVillage dataset, providing a visual interpretation of the features extracted using the EfficientNet V2-B1 model. This technique projects high-dimensional data into a two-dimensional space, enabling more intuitive understanding of feature distributions. The t-SNE is widely

used in Machine Learning and Computer Vision to visualize high-dimensional embeddings learned by deep networks. The t-SNE plot is indeed primar-ily a qualitative, exploratory visualization technique and not a rigorous statistical test for separability, it is commonly used to suggest or

visualize the presence of separable clusters [29]. T-SNE plot generation depends on several parameters and can vary with the number of iterations. So it is just a way for Initial Exploration of data for Quickly determining if data is inherently clustered before running supervised models. The t-SNE can project the feature vectors with hundreds or thousands of dimensions into 2D space. It illustrates the distribution of learned features and support interpretability of the feature space. In the current work it is used for initial investigation of the structure of the learned representation. But it does not mathematically prove separability.

Table 2

First order statistical analysis.

Feature	Value	Interpretation
Mean	−0.150	Near zero center
Variance	0.749	Moderate intensity spread means the dataset
		contains noticeable variation
Standard Deviation	0.843	Moderate image contrast of diseased
		leaves due to lesions and discoloration
Skewness	−0.502	Slightly left-skewed indicating more high-intensity pixels with some darker regions
Kurtosis	−0.361	Distribution is Flatter than normal, indicating the pixel intensities are spread across a wider range

Table 3

Second order statistical analysis.

Feature	Value	Interpretation
Contrast	6060.39	Very high contrast indicates strong intensity differences between neighboring pixels,
Correlation	0.466	Moderate correlation means neighboring pixels have a moderate linear relationship, suggesting a mixture of smooth regions and irregular diseased areas.

Energy	0.0589	Low energy indicates that the texture is not uniform, meaning the leaf surfaces have irregular patterns due to disease symptoms.
Homogeneity	0.1307	Low homogeneity suggests neighboring pixels are not very similar, which again indicates irregular textures caused by disease regions.

As shown in Fig. 9, the improved separability of features indicates that the dataset indeed comprises 15 distinct classes, with each color corresponding to a specific disease category. The t-SNE analysis was conducted with the following hyperparameters: PCA-based initializa-tion, perplexity = 30, nearest neighbors = 90, early exaggeration = 12, metric = cosine, theta = 0.5, learning rate = 218.42, number of iterations = 250, and random seed = 42, ensuring full reproducibility. It is important to note that the axes represent abstract mathematical pro-jections without physical meaning; only the relative positions of points are significant, while absolute coordinates carry no interpretation (See Fig. 9).

1. *Description of the experimental setup procedure*

Hardware and Software Environment: All experiments were con-ducted on a workstation equipped with an Intel Core i7 11th Gen (11800H) processor, 16 GB of RAM, and a 1 TB SSD. To accelerate the deep learning computations, an NVIDIA GeForce RTX GPU with 8 GB of VRAM was utilized. To ensure a fair and consistent comparison across all models, a unified development platform was maintained throughout the study.

Experimental Design and Data Pre-processing : To ensure ar-chitectural consistency and computational efficiency, all input images were standardized to a 224 × 224 pixel resolution across all Efficient-NetV2 variants. The training pipeline utilized a robust data augmenta-tion strategy to enhance model generalization, incorporating random

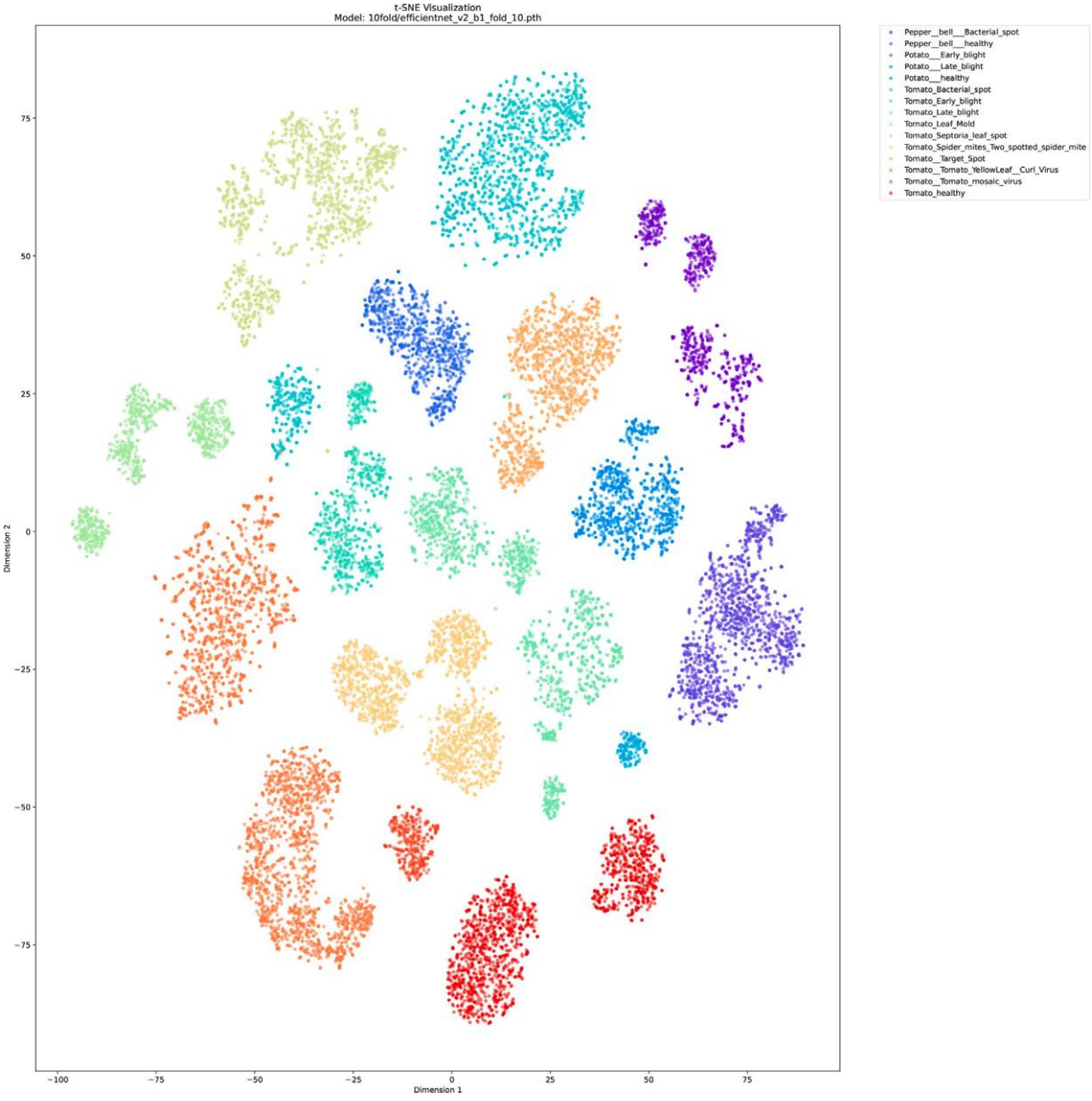
horizontal flips, rotations within a 15◦ range, and color jittering—

adjusting brightness, contrast, and hue by ±20%. Additionally, a ran-dom resized cropping strategy (scale 0.8 to 1.0) was implemented to ensure the model remained invariant to scale and object positioning.

Training Protocol and Hyperparameters: The proposed models were trained over 10 epochs with a mini-batch size of 8. The final reported results (in Tables 6–8) are based on 10-Fold Cross-Validation This was performed on the full dataset (excluding a separate, unseen test set) to ensure the results are robust and not biased by a single random split. We report the Average accuracy and standard deviation across the 10 folds. Again, we compared the K = 10 fold with K = 5 fold results. This approach ensures that the model is not overfitting on the training data and that the model's performance is robust across

different datasets. The training process utilized the Adam optimizer with a constant learning rate of 0.001 and dropout value 0.2. A sum-mary of the specific hyperparameter configurations is provided in Table

4. Furthermore, Table 5 delineates the architectural depth of each model, highlighting the total number of trainable layers where weight updates were permitted during the fine-tuning phase. The selection of the last 8(or 11 or 26) layers was based on established Transfer Learning heuristics. EfficientNetV2-B1 and EfficientNetV2-M are archi-tecturally different models, so it is normal that the ‘‘final block’’ (last MBConv/fused-MBConv stage before the head) has different numbers of repeated layers; B1 having 11 and M having 26 in that stage, is consistent with the idea that larger variants add depth in later stages to increase capacity while keeping earlier stages relatively small. To maintain methodological consistency across varying model scales, the number of unfrozen layers was scaled proportionally to the total depth of the architecture. While the EfficientNetV2-B1 utilized an 11-layer unfreezing window, the EfficientNetV2-M required a 26-layer window. This adjustment ensures that in both cases, the entirety of the final MB-Conv block and the high-dimensional convolutional head are subjected to task-specific fine-tuning, representing approximately 3.5% of the total network depth. To achieve an optimal balance between transfer learning stability and task-specific adaptation, we adopted an empirical progressive unfreezing strategy. Specifically, the final eleven layers of the EfficientNetV2-B1 architecture were unfrozen for fine-tuning. This allowed the model to recalibrate its high-level semantic descriptors for agricultural pathologies while maintaining the robust, generalized low-level edge detectors localized in the earlier stages of the backbone. In the strictest sense of the EfficientNetV2 paper (by Tan, Le), Progres-sive Learning also involves Adaptive Regularization (changing dropout rates and image sizes during training). However, in common research nomenclature, the strategy of Progressive Layer Unfreezing is a valid and widely accepted form of progressive transfer learning. It is particu-larly ‘‘empirical’’ because we likely tested unfreezing blocks and found that last two blocks unfrozen provided the best convergence without overfitting. We provided the evidence by comparing different parame-ters between full-fine tuned model and partial fine-tuned model in Figs. 13 and 14. The classification accuracy for 15 classes of leaf images from the PlantVillage dataset is shown in Table 6. EfficientNetV2-B1 achieves



/ig. 9. Visualization of t-SNE chart for 15 class dataset.

the highest average accuracy of 99.71%. The performance of vari-ous FT-EfficientNetV2 configurations on the real-time drone-captured dataset is detailed in Table 7. The experimental results indicate that efficientNetV2-B1 and efficientNetV2-M achieve comparable accuracy performance, but comparing the trainable parameter size, efficient-NetV2-B1 is the best choice which is given in Tables 6 and 7. As demonstrated in Table 6, the fine-tuned EfficientNetV2-B1 model out-performs current state-of-the-art methods in plant disease classification. Consequently, a comprehensive class-wise performance analysis was conducted for this model, evaluating Precision, Recall, F1-score, and Accuracy, as detailed in Table 11. The analysis reveals a high degree of diagnostic consistency, with class-specific accuracies ranging between 96.93% and 99.85%. These results underscore the model's robustness in identifying a wide spectrum of pathological symptoms across different plant species. The Fig. 10 illustrates the performance metrics of plant village dataset across 10 cross-validation folds, including Accuracy, F1-score, Precision, and Recall. The fold-wise evaluation demonstrates consistently high accuracy, precision, recall, and F1-score across all 10 folds, with an average F1-score of 0.994, confirming the robustness and generalization capability of the proposed model. The experimental

pipeline was strictly organized in the following order: (1) the dataset was first split into 10-fold cross-validation; (2) within each fold, the training subset was augmented after the split, while the validation subset remained untouched; and (3) feature extraction, normalization, and classifier training were performed exclusively using the training data of each fold. No augmented or derived samples were shared across folds or with the unseen test set. This ensures that validation and test samples were never exposed to the model during training. We have now added a detailed stepwise explanation in the revised manuscript to improve transparency and reproducibility. Pipelined diagram Fig. 11 is provided for reference. The kfoldcrossvalidation() function performs K-fold cross-validation to evaluate our deep learning model in a robust and research-grade manner. First, the dataset is split into K equal parts using KFold, ensuring shuffled and reproducible splits. For each fold, a different portion of the data is used for validation while the remaining data is used for training. Separate ImageFolder datasets are created with appropriate transforms (data augmentation for training and stan-dard preprocessing for validation), and SubsetRandomSampler ensures only the correct indices are used in each split. A fresh EfficientNet model is initialized for every fold to prevent data leakage between

Table 4

Hyper-parameter summary for Fine-tuned effi-cientNetV2 model.

Table 7

Performance of improved EfficientNetV2 model on drone type dataset.

Model	Av.Acc	Av.Recall	Av.Precision	Av.F1 score			
Hyper parameters	Details		efficientNetV2-B0	98.09%	98.85	98.1	98.34
input image size	224 × 224		efficientNetV2-B1	99.03%	99.03	99.01	99.10
Mini batch size	8		efficientNetV2-B2	98.22%	98.21	98.02	98.22
Learning rate(LR)	0.001		efficientNetV2-B3	98.32%	98.36	98.25	98.31
Optimizer	Adam		efficientNetV2-S	97.47%	97.45	97.43	97.46
Epoch	10		efficientNetV2-M	99.35%	99.35	99.35	99.35
dropout	0.2		efficientNetV2-L	98.74%	98.77	98.70	98.77

Table 5

Trainable layer details for improved EfficintNetV2 model.

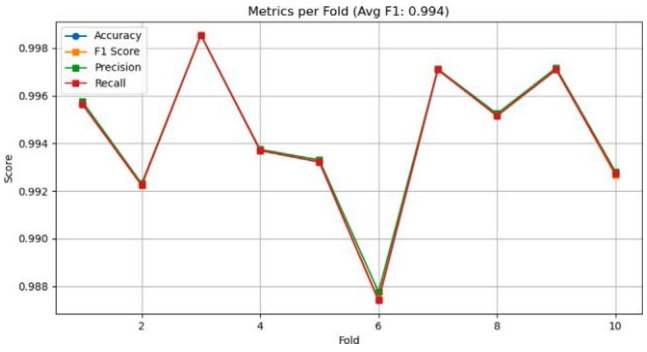
EfficientNetV2 model	Trainable layers for final two blocks
EfficientNetV2-B0(0–271 layers)	264–271
EfficientNetV2-B1(0–335 layers)	325–335
EfficientNetV2-B2(0–350 layers)	340-350
EfficientNetV2-B3(0–410 layers)	400–410
EfficientNetV2-S(0–514 layers)	504–514
EfficientNetV2-M(0–741 layers)	716–741
EfficientNetV2-L(0–1029 layers)	1019–1029

Table 6

Performance of improved EfficientNetV2 model on plant village dataset.

EfficientNet model	Av.Accuracy	Av.F1 score	Av.precision	Av.Recall
efficientNetv2-B0	98.1	98.12	98.86	98.79
efficientNetv2-B1	**99.71**	99.43	99.51	99.47
efficientNetv2-B2	99.55	99.62	99.59	98.58
efficientNetv2-B3	99.62	99.55	99.66	99.26

efficientNetv2-L	99.65	99.61	99.65	99.67
efficientNetv2-M	99.67	99.67	99.57	99.68
efficientNetv2-S	96.93	96.93	96.87	96.78



/ig. 10. Performance metrics plot for plant village dataset using Efficient-NetV2-B1.

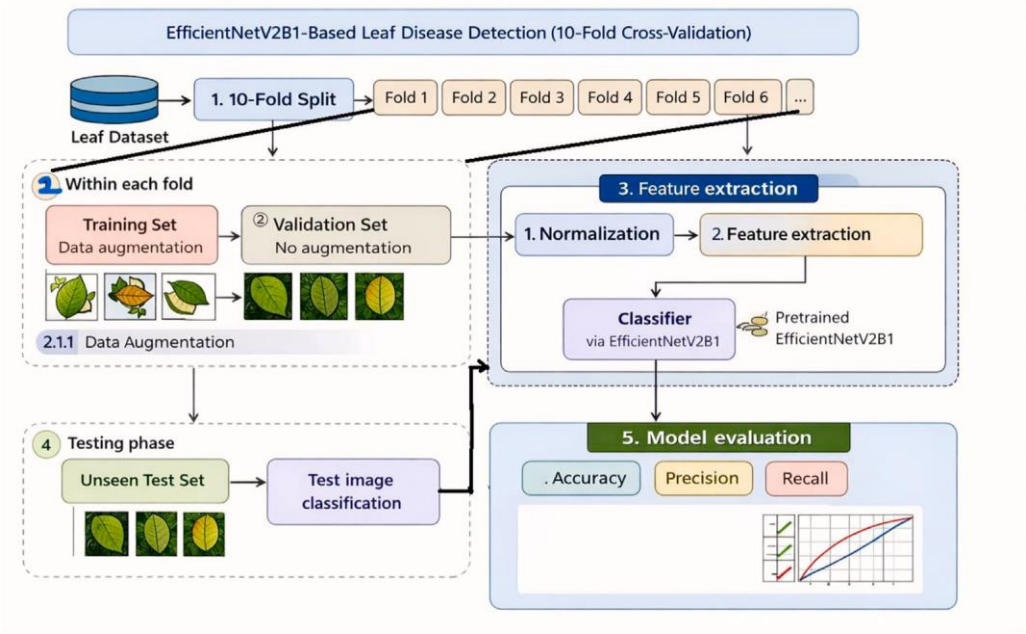
folds. During training, epoch-wise training and validation losses and accuracies are recorded, and a learning rate scheduler adjusts the learning rate(reduces by a factor) if validation loss stops improving within a specified number of epochs, with the help of the factor and patience command. The average validation accuracy is calculated by saving the validation accuracy of each fold. Finally, all fold metrics are stored in fold-results, which allows further analysis such as plotting learning curves, computing mean ± standard deviation, and performing statistical tests like a one-way ANOVA test to assess model verification and generalization performance.

We discovered a high accuracy range for all efficientNetV2 models from Tables 6 and 7, which allows us to determine additional statistical factors that support the performance analyses listed below. 1. We im-plemented a hold-out test set + 10-fold cross-validation strategy, which is a robust evaluation protocol for deep learning models. The entire dataset was divided into 85% for training and validation, where 10-fold cross-validation was applied, and the remaining 15% was reserved as

an independent test set. This independent dataset is never used during training or validation. It is evaluated only after all folds are completed. Within the 85% train-validation portion, 90% is training data, and the rest is validation data. A different validation subset was created, and the process was repeated for 10 folds. The training set was augmented, but the validation subset was not. So 90% of the training validation data, or 76.5% of the whole dataset, is given for the training process. Rest 10% of that dataset, or 8.5% of the whole dataset, is used for validation. The 10-fold accuracy result we got as 99.50 ± .16%. 10-fold results should be ignored, and the whole train–validation(85% dataset) is retrained. Evaluation is done once with 15% of the test data. whereas for the unseen test dataset, accuracy is 98.93%, confirming strong gen-eralization capability. Additional statistical metrics further validated the effectiveness of the model, including a macro-averaged precision of 98.74%, recall of 98.86%, F1-score of 98.78%, balanced accuracy of 98.86%, Cohen’s Kappa coefficient of 0.9884, and an

AUC-ROC score of 1.000. The high Kappa value indicates near-perfect agree-ment between predicted and true labels, while the balanced accuracy demonstrates consistent performance across all disease categories. The perfect AUC-ROC score highlights the model's excellent discriminative ability in distinguishing among the 15 plant disease classes. Similarly, for the drone-type dataset, the model's final unseen test accuracy was 99.85%, and its mean cross-validation accuracy was 99.82% ± 0.24%. Furthermore, it demonstrated outstanding classification performance, robustness, and generalization power with a balanced accuracy of 99.85%, a Cohen's Kappa score of 0.9978, and an AUC-ROC of 1.0000.

1. Independent-dataset validation: We tested our model on the PlantVillage 38 class dataset (54,913 field images), achieving 99.09% accuracy. This demonstrates generalizability beyond dataset-specific artifacts.
2. ANOVA results: We performed a few tests for model verifi-cation, which are given below in Table 8. We experimented with the ANOVA test for 4 models, showing better results; i.e., Efficient-NetV2-B0, EfficientNetV2-B1, EfficientNetV2-M, and EfficientNetV2-s, with 10-fold results. With a mean validation accuracy of 99.37% and a low standard deviation of 0.59, Effnetv2B1 produced the greatest overall performance across ten folds, demonstrating robust and reli-able generalization. With a mean validation accuracy of 99.19% and an even lower standard deviation of 0.49, Effnetv2-s trailed closely, demonstrating strong consistency over folds. With a mean validation accuracy of 99.02%, Effnetv2-M likewise did well, but its variation was marginally greater than that of B0 and S. Effnetv2-B0, on the other hand, had the most variability and the lowest mean validation accuracy of 98.74%, primarily because of a discernible decline in Fold 7. In terms of accuracy and consistency, Effnetv2-B1 is the model that performs the best overall out of the four. One-way ANOVA across 10-fold cross-validation showed no significant difference between EfficientNetv2 variants ($F = 0.56$, $p = 0.647$), confirming consistent performance across architectures. 4. Normality and variance test:Shapiro–Wilk test determines if the distribution of our data is normal. Its null hypothesis is that the sample is representative of a normal population; normality is rejected if the p-value is less than 0.05 and accepted if the p-value is more than 0.05. Similarly, Levene's test is typically used to determine variance significance. The null hypothesis is that the variances of the



/ig. 11. Pipeline diagram for elimination of data leakage using k-fold data splitting.

groups are equal; if the p-value is larger than 0.05, the variances can be considered homogeneous; if the p-value is less than 0.05, the variances are significantly different. The Shapiro–Wilk test shows that only EfficientNetv2-M looks normally distributed, while the others do not. This means the normality assumption of ordinary one-way ANOVA is not fully satisfied. However, Levene's test has p = 0.5983, so the homogeneity of variance assumption is satisfied. According to our previous findings, Levene's test was not significant, indicating that the variances were homogeneous. The majority of EfficientNetv2 groups failed the Shapiro–Wilk test, indicating that normality is not fully satisfied. This indicates that while the variance assumption of ANOVA is acceptable, the normality assumption is not working. 5. Kruskal–Wallis H-test: The models' fold-wise accuracies were compared using a Kruskal–Wallis H-test. The observed performance differences were not statistically significant, as the test revealed no significant difference between the groups (H = 2.338, p = 0.505). The fold-wise accuracies of the models are similar enough that this test cannot claim one model is better than the others. 6. A Tukey HSD post hoc test was performed to compare the EfficientNetv2 variants pairwise. The results showed in Table 9 no significant pairwise differences among EffNetV2-b0, b1, M, and S, as all adjusted p-values were greater than 0.05 and all confidence intervals included zero. The ''Mean-Diff'' column displays the average performance difference between two models, and the ad-justed p-value (p-adj) after multiple comparison correction indicates statistical significance. After accounting for repeated comparisons, this is the probability value. Confidence intervals are displayed at the lower and upper bounds. If p-adj < 0.05 and the confidence interval excludes zero, the null hypothesis is rejected, indicating a statistically significant difference. The findings showed that, at the 5% significance level, none of the pairwise comparisons were statistically significant (all adjusted p-values > 0.05). Additionally, all mean differences had 95% confidence intervals that included zero, indicating significant model performance overlap. These results support the ANOVA/Kruskal–Wallis results and show that random variation rather than actual differences in model performance is probably responsible for the observed differences in classification accuracy among EfficientNetV2-B0, EfficientNetV2-B1, EfficientNetV2-S, and EfficientNetV2-M.

The Fig. 12 shows fold-wise dispersion for the drone-type dataset with CV K = 10, which shows the validation accuracy slightly exceeds

Table 8

Comparison of EfficientNetV2 models based on training and validation perfor-mance.

Model	Mean val (%)	Std Val	Mean train (%)	Std train
EffNetV2-B0	98.74	1.67	97.24	2.16
EffNetV2-B1	99.37	0.59	98.22	0.49
EffNetV2-S	99.19	0.49	98.17	0.37

EffNetV2-M	99.02	1.30	98.26	0.27

Table 9

Tukey HSD Post-hoc test results for validation accuracy.

Group 1	Group 2	Mean Diff.	p-adj Lower	Upper	Reject
EffNetV2-B0	EffNetV2-B1	0.6250	0.6055 – 0.7314	1.9814	False
EffNetV2-B0	EffNetV2-M	0.2790	0.9449 – 1.0774	1.6354	False
EffNetV2-B0	EffNetV2-S	0.4460	0.8123 – 0.9104	1.8024	False
EffNetV2-B1	EffNetV2-M	−0.3460	0.9014 – 1.7024	1.0104	False
EffNetV2-B1	EffNetV2-S	−0.1790	0.9844 – 1.5354	1.1774	False
EffNetV2-M	EffNetV2-S	0.1670	0.9872 – 1.1894	1.5234	False

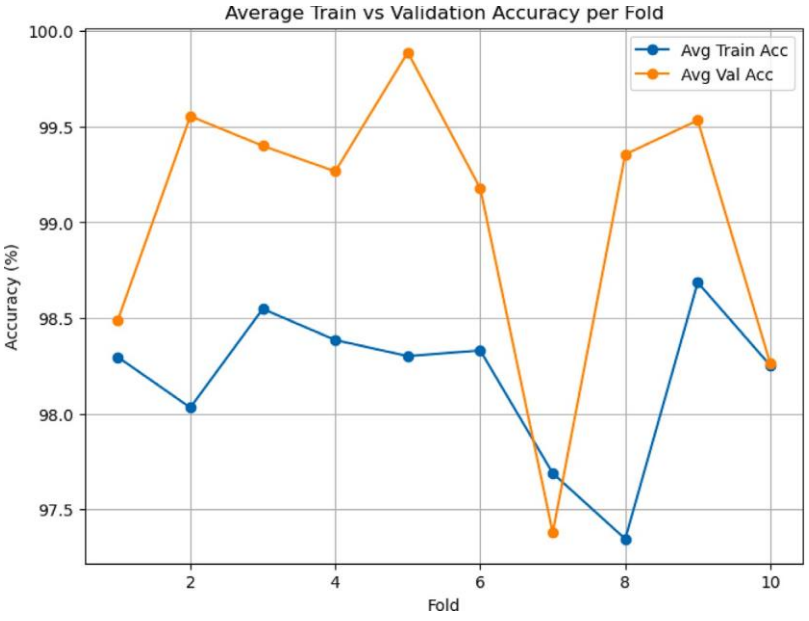
Table 10

Fold-wise performance evaluation for drone dataset EfficientNetV2-B1 model.

Fold	Train. Acc	Val. acc	Observation
1	~98.3	~98.4	Balanced performance
2	~98.0	~99.5	Large improvement in validation
3	~98.5	~99.4	Stable performance

4	~98.4	~99.3	Consistent accuracy
5	~98.3	~99.7	Highest validation accuracy
6	~98.3	~99.1	Stable
7	~97.7	~97.6	Lowest validation accuracy
8	~97.3	~99.4	validation high
9	~98.7	~99.6	Strong performance
10	~98.3	~98.3	Balanced

the training accuracy, possibly due to regularization effects and cross-validation data distribution. The fold-wise result is analyzed in Table 10 for drone type dataset. The convergence curve plots the performance metric (loss or accuracy) versus training iterations (folds). So mathe-matically:f(e) = "Loss or Accuracy at epoch e. Where: e = epoch number f(e) = model performance The average convergence curve illustrates the



/ig. 12. /old-wise dispersion convergence graph for Drone-type dataset K = 10 using EfficientNetV2-B1.

Table 11

Disease-wise performance for EfficientNetV2-B1 model.

PLANT DISEASE NAME	ACC	F1	PREC	RECALL
Bellpepper Bact.spot	0.995523	0.995	0.9954	0.9952
Bellpepper healthy	0.992236	0.9923	0.9927	0.993
Bellpepper early blight	0.998556	0.998	0.9985	0.9982
potato late blight	0.993105	0.9935	0.9943	0.9936
potato healthy	0.997236	0.987952	0.98723	0.996
Tomato bact.spot	0.998523	0.989691	0.979592	0.998
Tomato early blight	0.997396	0.9978	0.9983	0.9987
Tomato late blight	0.99523	0.996	0.995	0.9956
Tomato leaf mold	0.998147	0.9984	0,9986	0.99
Tomato septoria leaf	0.998523	1	1	1
Tomato spider mites	0.997273	0.997	0.9986	0.997
Tomato target spot	0.998515	1	1	1
Tomato healthy	0.998104	0.978723	1	0.958333
Tomato yellowleafcurlvirus	0.9981	0.9985	0.9976	0.9982
Tomato mosaic virus	0.9986	0.998	0.9987	0.992

learning progression of the model across 10 epochs under 10-fold cross-validation. The training accuracy gradually increases and stabilizes around 98%–98.7%, indicating steady parameter optimization. The validation accuracy remains consistently high (above 99%) across all epochs, with minor fluctuations. The close alignment between training and validation accuracy suggests minimal overfitting and strong gener-alization performance. The absence of large oscillations in both curves indicates stable convergence behavior and an appropriate learning rate selection. Overall, the model converges efficiently within a few epochs while maintaining high classification accuracy.

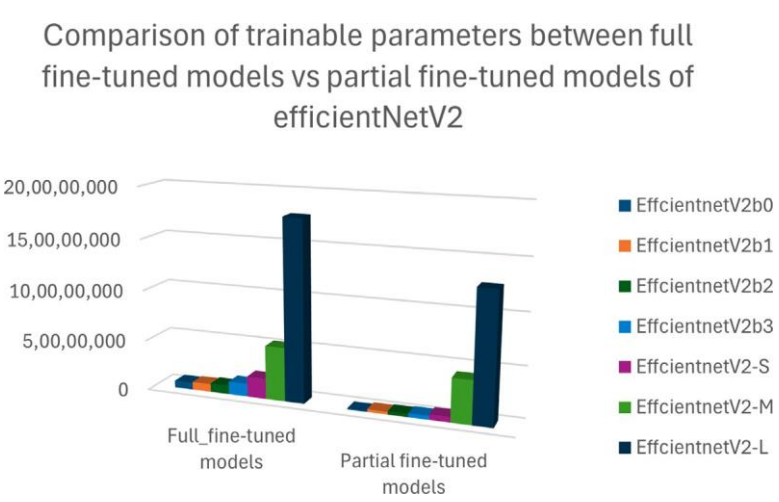
In particular, two configurations were assessed: (1) partial fine-tuning, in which only the final blocks were unfrozen while previous layers remained frozen, and (2) full fine-tuning, in which all network parameters were

updated during training. Training duration, the quan-tity of trainable parameters, convergence behavior, and classification performance are all compared. The findings demonstrate that partial fine-tuning preserves comparable classification accuracy while drasti-cally reducing the number of trainable parameters and training time. This behavior is expected since deeper layers of pretrained convolu-tional networks learn task-specific representations, while early layers capture general low-level visual elements like edges and textures that are transferable across tasks. As a result, the model can specialize for

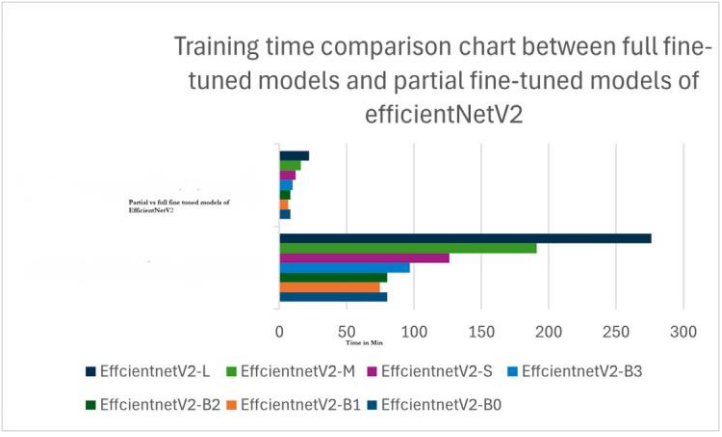
the target dataset while maintaining the valuable pretrained charac-teristics by simply changing the final blocks. A structured ablation study comparing full and partial fine-tuning has been given to fur-ther support this design decision (Figs. 13 and 14). Figs. 13 and 14 demonstrate that partial fine-tuning reduces the number of trainable parameters compared with full training, leading to lower computational cost and faster training time while still allowing the model to adapt high-level features to the target dataset. This strategy is particularly beneficial when working with relatively small datasets, as it balances model adaptability and computational efficiency. Training duration scales predictably with model complexity in Fig. 14. Full fine-tuning (all 68M–528M parameters active) takes 74–276 min, whereas partial unfreezing (final two blocks + classifier head, 11M–55M parameters) reduces computation by 88%–92% to just 6–22 min. EfficientNetV2-B1 offers an optimal balance at 74.55 min full fine-tuned vs 6.40 min partially fine-tuned. Figs. 15 through 21 illustrate the accuracy and loss trajectories for the various EfficientNetV2 configurations. All ex-periments were conducted using the Adam optimizer and categorical cross-entropy loss, with a mini-batch size of 8 over 10 epochs. Notably, the learning curves demonstrate rapid convergence, stabilizing as early as the 8th epoch. This indicates a highly efficient training process where the model quickly optimizes its weights for the plant disease features. All evaluated models exhibited exceptional performance, with accuracies ranging from 96.93% to 99.71%. Among these, the Efficient-NetV2-B1 variant emerged as the top performer, surpassing existing state-of-the-art methods. Evenif EfficientNetV2-M and V2-B1 show ac-curacy in the same range but due to high computational complexity, EfficientNetV2-B1 is the best choice.

1. *Grad-CAM*

The interpretability of the model is established through Gradient-Weighted Class Activation Mapping (Grad-CAM), which visualizes the decision-making process of the CNN-based classifier. Grad-CAM gen-erates heatmaps that highlight the specific spatial regions within an image that most significantly influence the model's prediction. In Fig. 22, a few samples of original diseased leaf images, with the correspond-ing Grad-CAM images, are provided for reference. These heatmaps demonstrate that the model focuses on pathological features. Since the datasets used in this study do not contain expert-annotated lesion masks, it was not possible to compute localization metrics such as Intersection over Union (IoU) or Dice coefficient. Therefore, it is clear



/ig. 13. Comparison of trainable parameters between full fine-tuned and partial fine-tuned efficientNet models for drone dataset.



/ig. 14. Comparison of training time between full fine-tuned and partial fine-tuned efficientNet models for drone dataset.

Table 12

Performance comparison of proposed model with state-of-art mode.

Author	Dataset	Method applied	Accuracy
[9]	plantvil.	GoogleNet	99.35%
[21]	plantvil.	EfficintNetB4	99.97%
[30]	plantvil.	ResNet50	96.62%
[31]	plantvil.	ResNet50 and ResNet101	99.8%
[32]	plantvil.	Time efficient andlow cost model	95%
[33]	plantvil.	LBGM ,machine learning based model	94.39%
[34]	plantvil.	VGG-19 model followed by ViTTLMViT	98.81%

[19]	plantvil.	PDDNet-AE and PDDNet-LVE	96.6%&97.7%
Proposed	plantvil.	EfficientNetV2-B1 (improved)	99.71%
[3]	drone	ResNet50 + SVM	98.3%
[35]	drone	Dual thresholding +SVM	98.84%
[35]	drone	Dual thresholding +LS-SVM	98.87%
proposed	drone	EfficientNetV2-B1 (improved)	99.03%

that Grad-CAM is used solely for qualitative visualization of the model's attention regions rather than as a quantitative evaluation of lesion lo-calization performance. To make this section more rigorous, we briefly explained the mathematical flow. Grad-CAM uses the gradient infor-mation flowing into the last convolutional layer of the EfficientNetV2 backbone to assign importance values to each neuron. The resulting

1.

Discussion

1. We performed a few tests for model verification, which are given below. We experimented with the ANOVA test for 4 models, show-ing better results; i.e., EfficientNetV2-B0, EfficientNetV2-B1, Efficient-

heatmap is a localization map L^c

$\in \mathrm{R}^{u \times v}$ for any class c,

NetV2-M, and EfficientNetV2-s, with 10-fold results. We incorporated

calculated as:

$(\sum \quad)$

$Grad-CAM$

different statistical analyses performed across cross-validation folds be-tween the proposed framework and competing baseline architectures.

c

Grad-CAM

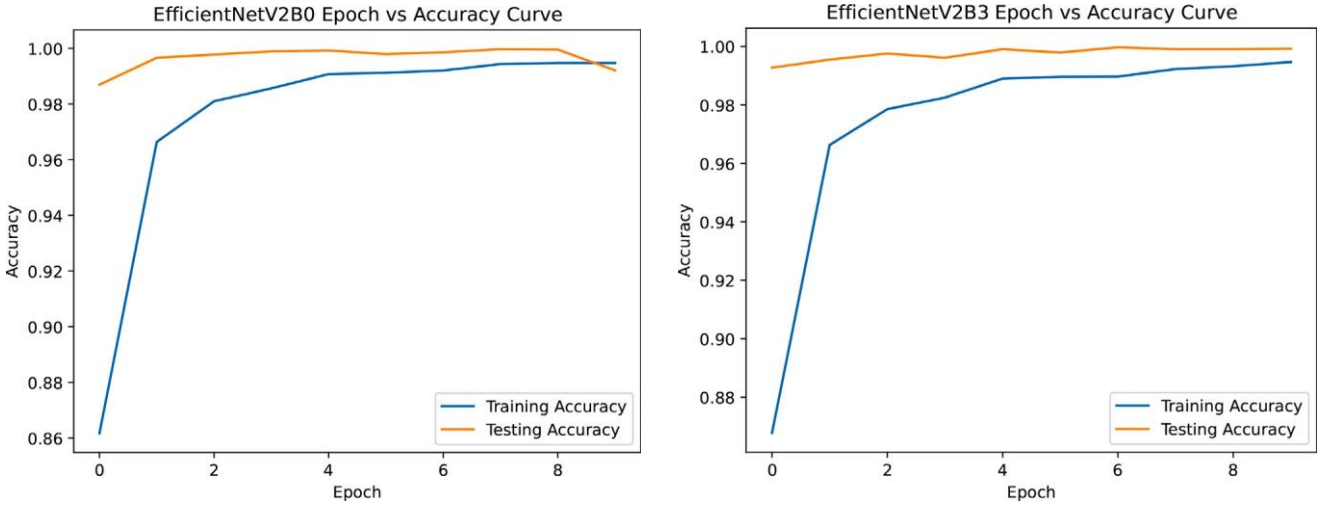
.

= ReLU

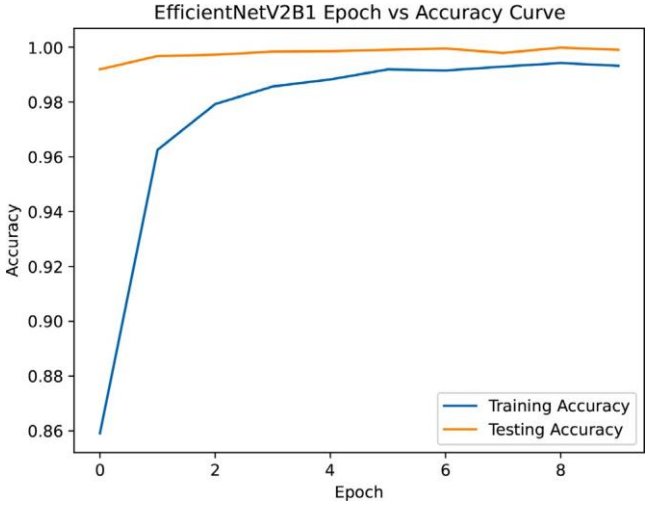
ac Ak

k

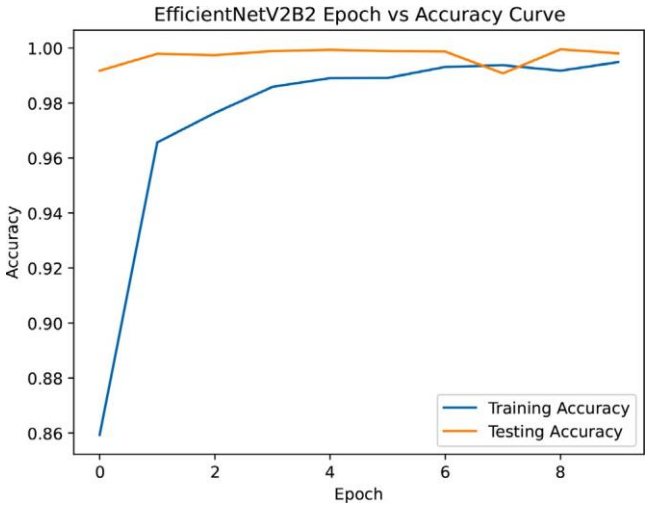
Specifically, ANOVA test, Tukey's HSD post hoc test, and Kruskal–Wallis H-test were conducted using fold-wise performance metrics to



/ig. 15. Accuracy graph for EfficientNetV2-B0.

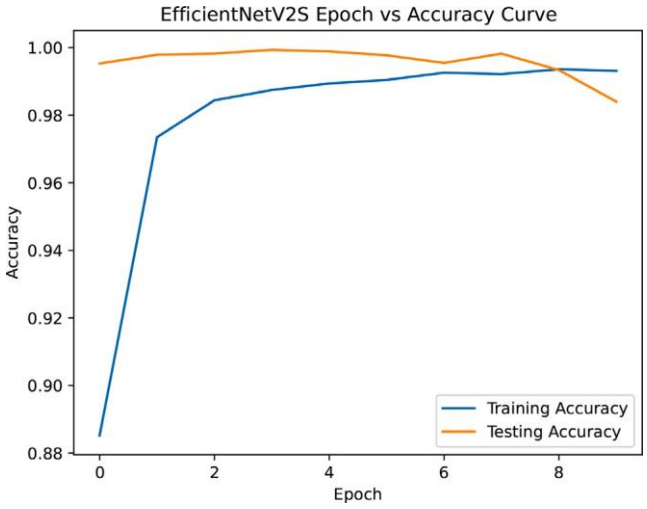


/ig. 16. Accuracy graph for EfficientNetV2-B1.

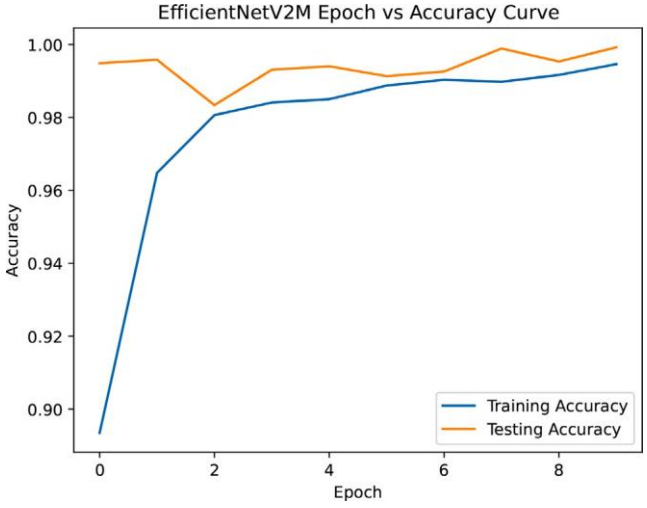


/ig. 17. Accuracy graph for EfficientNetV2-B2.

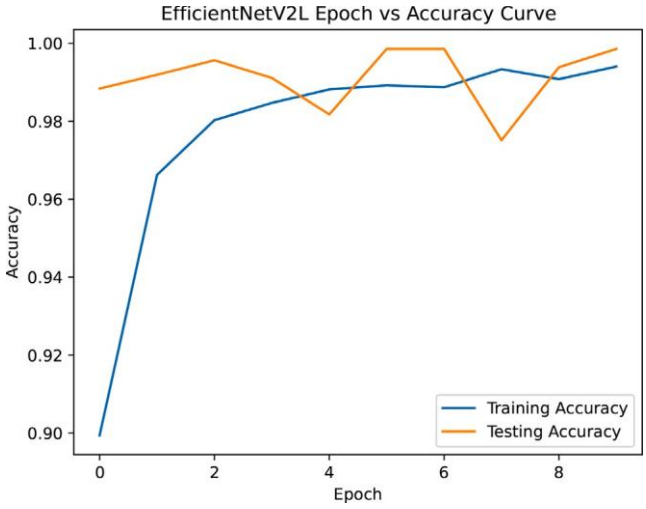
/ig. 18. Accuracy graph for EfficientNetV2-B3.



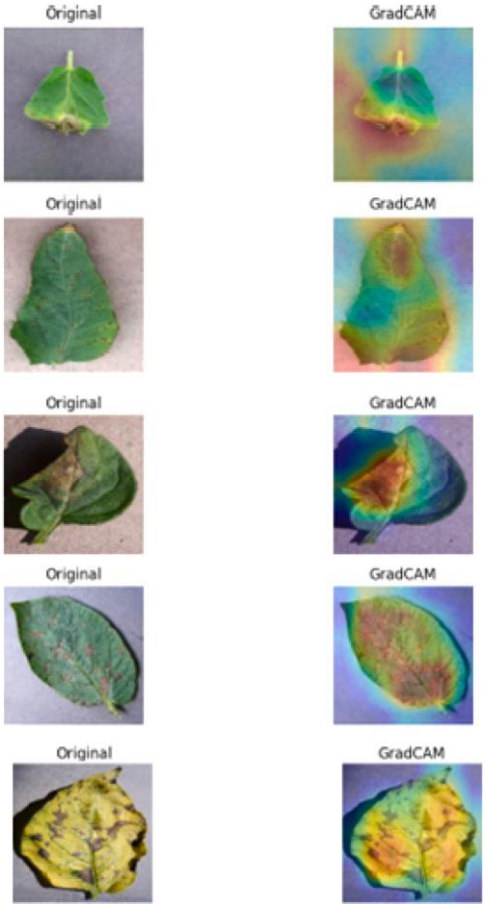
/ig. 19. Accuracy graph for EfficientNetV2-S.



/ig. 20. Accuracy graph for EfficientNetV2-M.



/ig. 21. Accuracy graph for EfficientNetV2-L.



/ig. 22. Grad-cam effect.

evaluate whether the observed improvements are statistically signif-icant. Additionally, mean and standard deviation values have been incorporated to better quantify result variability and reliability. These additions strengthen the statistical rigor of the evaluation and pro-vide more comprehensive evidence supporting the effectiveness of the proposed method. The results say there is no statistically significant difference among the EfficientNetv2 variants for either validation ac-curacy or training accuracy. In simple terms, the small differences

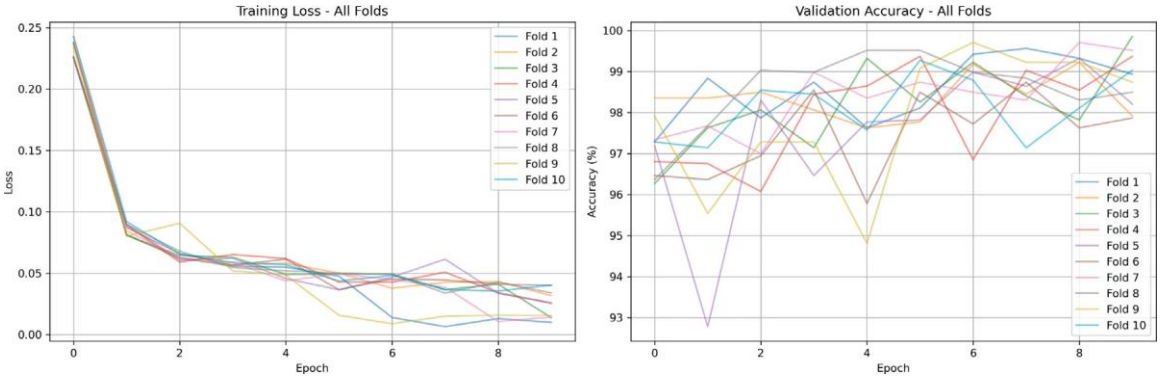
in mean accuracy that we see are not large enough, relative to the variability, to conclude that one model is truly better than the others.

1. A comparison of cross-validation at K = 10 with K = 5 is provided in Figs. 23 and 24. Similarly, for the drone type dataset, correspond-ing figures are provided in Figs. 12 and 25. 3. We agree that very high accuracies in deep learning-based agricultural image analysis may raise concerns related to dataset simplicity, limited variability, and the possibility of hidden correlations, particularly when relatively small drone-acquired datasets are involved. To mitigate these issues, several precautionary measures were adopted in the present work, including strict fold-wise data separation, post-splitting augmentation, random-ization during training, and evaluation using multiple statistical and performance metrics. Furthermore, the dataset contains variations in disease severity, leaf orientation, illumination conditions, and acqui-sition viewpoints to improve diversity. Nevertheless, we acknowledge that the dataset size and controlled acquisition conditions may still contribute to optimistic performance estimates.

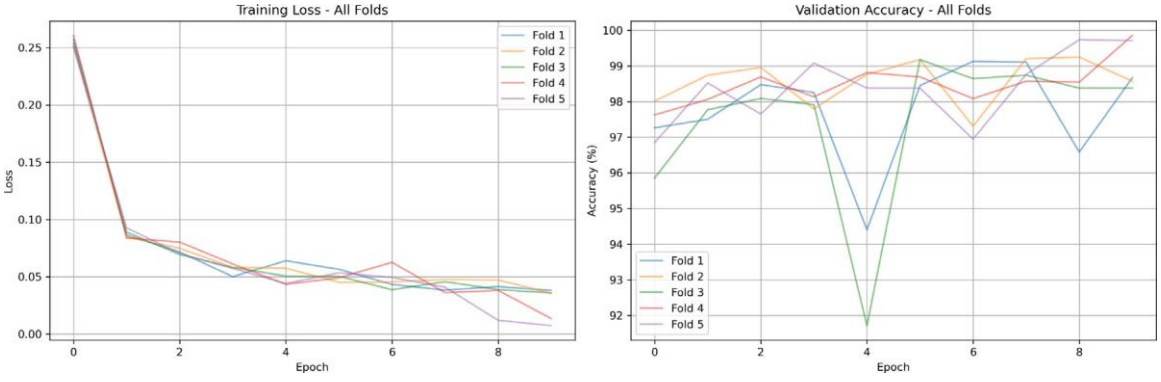
From Figs. 23 and 24, it is clear that the training loss and validation accuracy curves across five-fold and 10-fold demonstrate rapid conver-gence and stable generalization, with high accuracy achieved within the first eight epochs, confirming the efficiency and robustness of the fine-tuned EfficientNetV2-B1 model. While both 5-fold and 10-fold cross-validation demonstrate rapid convergence and high accuracy, the 10-fold setup provides smoother and more stable performance across folds, whereas the 5-fold setup shows slightly higher variability due to larger validation splits. Accuracy in both cases are nearly in same range. Average accuracy for K = 10 is 99.13% with standard deviation of 0.30%. We provided an ablation study regarding fine tuning of our pro-posed model. Without fine tuning the model we got 96.07% of accuracy but with fine tuning our result shines with 99.71%. Similarly, for drone type dataset the average accuracy is found to be 99.03% with standard deviation of 1.32%. In Figs. 12 and 25 both training and validation accuracies are high between 97%–99.5%. In both cases, validation accuracy is higher than training accuracy, suggesting strong classifica-tion performance, no overfitting, and good generalization capacity. To investigate such a high accuracy (mean of 99.03%), we conducted an additional analysis of the drone dataset characteristics, including class distribution, image variability, and intra-class similarity. The dataset comprises 3 classes: Bacterial blight, Blast, and Brownspot. The full dataset contains 4499 images, with 3599 images used for training and 900 for validation. Class distribution was quantified by counting the number of samples per class and computing the class-imbalance ratio (smallest class divided by largest class) and the Shannon entropy of class proportions. A Chi-square goodness-of-fit test was performed using the SciPy implementation to verify that the dataset was not significantly unbalanced across classes. The test compared the expected uniform distribution with the observed class frequencies. The results indicate that the class distribution does not significantly differ from a balanced distribution ($p > 0.05$). The obtained statistic ($\chi^2 = 4.48$, $p = 0.106$) suggests that the dataset is nearly balanced with only mild skew among the classes. Train – imbalance ratio: 0.92 and Train – entropy (bits):

1.58. Similarly, for validation data, Val – imbalance ratio: 0.81 and Val – entropy (bits): 1.57. Which is correct for a nearly balanced 3-class distribution i.e (log2(3)). The Grad-CAM visualizations suggest that the model predominantly attends to regions associated with class-discriminative features; however, the limitation is this

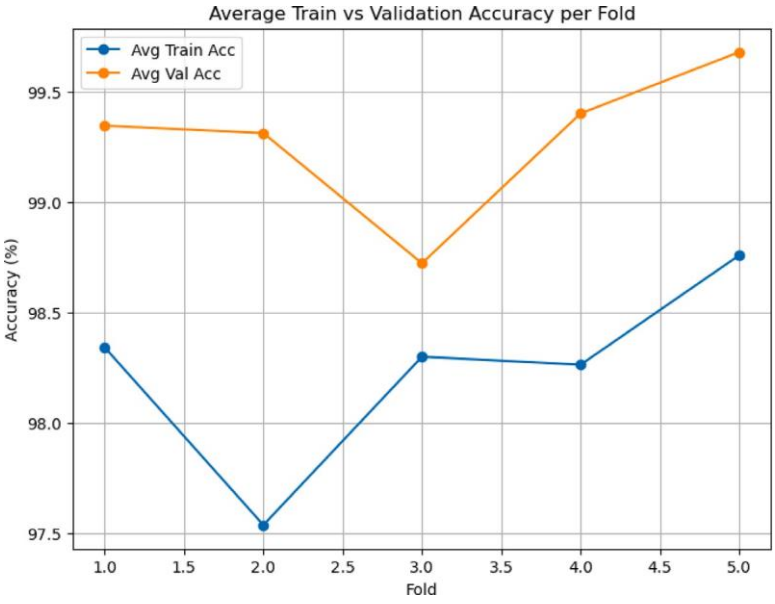
observation is qualitative and not quantitative due to the absence of pixel-level expert annotations in the dataset. In Table 12 the literature results are presented as contextual references rather than direct comparisons, with limitations of non-standardized. We experimental settings clearly acknowledged and superiority claims appropriately moderated. We en-sured that comparisons were restricted to studies using the same dataset to reduce variability in data sources. Table 12 presents a compara-tive analysis of the proposed approach against various state-of-the-art methods utilizing the same datasets. Several studies, including [9,19, 21,30–34], employed the PlantVillage dataset for their experiments.



/ig. 23. (a) training loss (b) validation accuracy for cv K = 10.



/ig. 24. (a) training loss (b) validation accuracy for cv K = 5.



/ig. 25. Average training-validation accuracy per fold for drone images cv K = 5.

For the drone-captured dataset, [3] and [35] reported significant re-sults. Notably, Vishnu et al. [30] achieved high performance using the ResNet-50 architecture. Similarly, Sachin et al. [31] reached an accu-racy of 99.8% using ResNet-50 and ResNet-101 models with the Adam optimizer ($\mathcal{LR} = 0.0001$). However, despite achieving near-perfect ac-curacy, the ResNet architecture entails high memory complexity due to its large parameter count—approximately 1.5 times greater than that of the EfficientNetV2-B1 model proposed here. Furthermore, while ResNet

relies on standard bottleneck blocks, our model utilizes Fused-MBConv (fused bottleneck) architecture, which offers superior training through-put. Additionally, Singh et al. [32] proposed a low-cost, time-efficient model but achieved a lower accuracy of 95% on the drone dataset, highlighting the superior balance of efficiency and precision found in our proposed FT-EfficientNetV2 framework. Previous studies have explored various approaches for rice disease detection with notable suc-cess. Acharya et al. [35] employed machine learning classifiers (SVM,

LS-SVM) and reported promising results. Sethy et al. [3] investigated hybrid architectures combining deep learning feature extractors with SVM classifiers, where ResNet-50+SVM achieved the highest accuracy of 98.3% on drone-captured imagery. While these results demonstrate the efficacy of hybrid approaches, the present study explores pure end-to-end deep learning with EfficientNetV2 to potentially surpass these benchmarks.

1. Conclusions and future work

The goal of this work is to employ the refined EfficientNetV2 architecture for automated disease diagnosis in leaf photos. While the current implementation is validated on the PlantVillage dataset, a thorough study of several EfficientNetV2 variations showed that EfficientNetV2-B1 obtained greater performance with an accuracy of

99.71. In order to provide real-time, in-field diagnosis, future research may concentrate on implementing the EfficientNet-based illness de-tection model on mobile phones and edge computing platforms. The inference time, throughput, and memory consumption of various Ef-ficientNet variations (B0–B7) may be examined through a thorough benchmarking research. The performance can be compared with other lightweight CNN architectures such as MobileNet, ShuffleNet to iden-tify the best trade-off between accuracy and latency. Further future research can focus on the robustness of the EfficientNet model under diverse field conditions, such as variations in lighting, background clutter, camera noise or low-resolution imagery. The model's stability and dependability in real-world deployment circumstances may be verified through field testing with photos taken straight from farms. Future research can investigate domain adaptation and transfer learn-ing algorithms for undiscovered crops and newly developing illnesses that can manage performance deterioration when moving the model to new datasets in order to improve generalization across crop species and geographic locations.

CRediT authorship contribution statement

Snehaprava Acharya: Writing – review & editing, Writing – orig-inal draft, Visualization, Validation, Software, Conceptualization. **T. Kar:** Writing – review & editing, Supervision, Project administra-tion, Investigation, Data curation, Conceptualization. **S. Pal:** Software, Methodology, Formal analysis. **P.K. Patra:** Writing – review & editing, Visualization, Project administration, Investigation, Formal analysis.

U.C. Samal: Writing – review & editing, Supervision, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing finan-cial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Raghu S., B. Navadagi Devanna, Hana Mathew S. Baite, K. Yadav Manoj, Prabhu Karthikeyan S. Rathinam, Keerthanaumapati Prajnapati, Prakash Ch. Rath, Emerging minor diseases of rice in India: Losses and management strategies, IntechOpen http://dx.doi.org/10.5772/intechopen.99898.
2. Jun Liu, X. Wang, Plant diseases and pest detection based on deep learning: A review, Plant Methods (2021) http://dx.doi.org/10.1186/S13007-021-00722-9.
3. P. Sethy, B. Negi, N. Barpanda, S. Behera, A.K. Rath, Measurement of disease severity of rice crop using machine learning and computative intelligence, Springer Briefs Appl. Sci. Technol. (2018) http://dx.doi.org/10.1007/978-981-10-6698-6_1.

4. B. Jangid, Rice disease detection using deep learning VGG-16 model and flask, Int. J. Sci. Res. Eng. Manag. (ISSN: 2582-3930) 7 (3) (2023) http://dx.doi.org/ 10.55041/IJSREM17874.

5. Chinna Gopi Simadri, Hari Kishan Kondaveeti, Automatic recognition of rice leaf diseases using transfer learning, Agronomy 13 (4) (2023) http://dx.doi.org/10. 3390/agronomy13040961.

6. Md. Mehandi Hasan, T. Rehman, A.E.M. Shahebuddin, Syed Md. Galib, M.R. Akhund, Md. Jasmit Uddin, Md. Alam Hossain, Enhancing rice crop manage-ment: Disease classification using convolutional neural networks and mobile application integration, Agriculture 13 (8) (2023) http://dx.doi.org/10.3390/ agriculture13081549.

7. G. Latif, E. Abdelhamid Sherif, R.E. Mallouhy, J. Alghazo, Deep learning utilization in agriculture: Detection of rice plant diseases using an improved CNN model, Plants 11 (2022) 2230, http://dx.doi.org/10.3390/plants11172230.

8. Andrew J., Jenifer Eunice R., Jude Hemanth D. Daniel Elena Popescu, Deep learning based leaf disease detection in crops using images for agricultural application, Agronomy (2022) http://dx.doi.org/10.3990/agronomy/12102.395.

9. Mohanty S.P., P. Huges David, Marcel Salathe, Using deep learning for image based plant disease detection, Frontiers in Plant Science (2016) http://dx.doi. org/10.3389/fpls.2016,01419.

10.]. Sunil C. K, C.D. Jaidhar, Nagammapatil " cardamom plant disease detection approach using efficientnev2, IEEE ACCESS 2022, http://dx.doi.org/10.1109/ ACCESS.2021.3138920.

11. A. Berfenati, Paola causin, Roberto Oberti, Giovanni Stefanello, " unsupervised deep learning techniques for automatic detection of plant diseases : reducing the need of manual labelling of plant images ", J. Math. Ind. (2023) http://dx.doi.org/10.1186/S13362-023-00133-6.

12. Walled Albattah, Ali Javed, Marriam Nawaz, Moina Masood, Saleh Albahli, " artificial intelligence-based drone system for multiclass plant disease detection using an improved efficient convolutional neural network, Sustain. Intell. Phy-toprotection, A Sect. J. Front. Plant Sci. (2022) http://dx.doi.org/10.3389/fppls. 2022.808380.

13. Yuanboye, HoukuiZhoze, Huimin Yu, Haoji Hu, Guangun Zhang, Jungho Hu, Tolo He, "' an improved efficientnetv2 model based on visual attention mecha-nism : application to identification of cassava disease ", Hindawi, Comput. Intell. Neuro Sci. (2022) 1569911, http://dx.doi.org/10.1155/2022/1569911.

14. R. Sujatha, Jyotirmay Chatterjee, N.Z. Shanjhi, Sarfraz Nawaz Brohi, " perfor-mance of deep learning vs machine learning in plant leaf disease detection ", Microprocess. Microsyst. 80 (2021) 103615, http://dx.doi.org/10.1016/j.micro.2020.103615.

15. Hongliang Guan, Chen Fu, Guangyuan Zhang, Kefeng Li, Peng Wang, Zhenfang Zhu, " A light Weight Model for Efficient Identification of Plant Diseases and Pests based on Deep Learning ". Front. Plant Sci.. 14. 1227011. http://dx.doi. org/10.3389/fpls.2023.1227011.

16. Jhunde Chen, Jinxiu Chen, Defu Zhang, Y. Y. Sun, A. NanehKaran, Using deep transfer learning for image based plant disease identification, J. Comput. Electron. Agric. 173 (2020) 105393, http://dx.doi.org/10.1016/j.compag.2020.105393.

17. Ananda S. Paymode, B. Malode Nandana, Transfer learning for multi crop leaf image classification using convolutional neural network VGG, J. Artif. Intell. Agric. 6 (2022) 23–33, http://dx.doi.org/10.1016/j.aiia.2021.12.002.

18. Mehdhar S.A.M. Al-Gaashani, Nagwan Abdel Samee, Reem Alkanhel, Ghada Atteia, A. Abdallah Hanaa, Asadulla Ashurav, Mohd saleh Ali Muthama, Deep transfer learning with gravitational search algorithm for enhanced plant disease classification, Heliyon 10 (2024) e28967, http://dx.doi.org/10.1016/j.hliyon. 2024.e28967.

19. Wasswa Shafir, Ali Jufail, C. DeSilva Liyanage, Rosyzie Anna Awg Haji Mohd. Apong, Using transfer learning based plant disease classification and detection for sustainable agriculture, J. BMC Plant Biol. 24 (2024) 136, http://dx.doi.org/ 10.1186/s12870-024-048257.

20. Shahoveisi Feroshteh, Hamed Taheri Gorji, Seyedali Hosseninad, Samuel markell, Fartash Vasefi, Application of image processing and transfer learning for the detection of rust disease, J. Sci. Rep. 13 (2023) 5133, http://dx.doi.org/10.1038/ s41598-023-31942-9.

21. U. Atila, M. Ucar, K. Akyal, E. Ucar, Plant Leaf Disease Classification using EfficientNet Deep Learning Model. J. Ecol. Inform. http://dx.doi.org/10.1016/ j.ecoinf.2020.101182.

22. S. Aboelenin, F.A. Elbasheer, M.M. Eltoukhy, W.M. El-Elady, K.M. Hosny, A hybrid framework for plant leaf disease detection using CNNs and vision transformer, Complex Intell. Syst. 11 (2025) 142, http://dx.doi.org/10.1007/ s40747-024-01764-x.

23. Y. Song, C. Yang, DS FusionNet:Dynamic Dual Stream Fusion with Bidirectional Knowledge Distillation for Plant Disease Recognition. http://www.arxiv.org/ 2504.20948. arXiv:2504.20948.

24. P. kour, A.M. Mishra, N. Goyal, S.K. Gupta, A. Shankar, W. viriyasitavat, " a novel hybrid CNN methodology for automated leaf disease detection and classification, Expert. Sysstem http://dx.doi.org/10.1111/exsy.13543

25. M. Aggarwal, V. Khullar, N. Goyal, T.A. Prola, Resource Efficient Federated Learning over IoAT for Rice Leaf Disease Detection. Comput. Electron. Agric.. http://dx.doi.org/10.1016/j.compag.2024.109001.

26. M. Aggarwal, N. Goyal, V. Khullar, "Exploring classification of rice leaf diseases using machine learning and deep learning, in: 2023 3rd InternationalConference on Innovative Practices in Technology and Management(ICIPTM), 2023, http://dx.doi.org/10.1109/ICIPTM57143.2023.10117854.

27. Tan M., Le. Quoc, " efficientnetv2: smaller models and faster thinking ", 2021, http://dx.doi.org/10.48550/ARXIV.2104.00298.

28. M. Tan, Q.V. Le, Efficientnet: rethinking model scaling for convolutional neural network, in: Proceedings of the 36th International Conference on Machine Learning (ICML, 2019, http://dx.doi.org/10.48550/arXiv.1905.11946.

29. L. van der Maaten, G. Hinton, Visualizing data using t-SNE, J. Mach. Learn. Res. 9 (2008) 2579–2605.

30. P. Vishnu Raja, K. Sangeetha, Ninisha B.A., M. Samiksha, S.S. Sanjutha, Convo-lutional neural network based classification and detection of plant diseases, in: 6th International Conference on Computing Methodologies and Communications, ICCMC, 2022, pp. 1484–1488, http://dx.doi.org/10.1109/ICCMC53470.2029753846.

31. Sachin Dahiya, Tarun Gulati, Dushyant Gupta, Performance analysis of deep learning architecture for plant leaves disease detection, J. Meas.: Sensors 24 (2022) 100581, http://dx.doi.org/10.1016/j.measen.2022.100581.

32. N. Singh, S. Singh, Time efficient and low-cost neural network to detect plant disease on leaves and reduce food loss and waste, J. Emerg. Investig. 6 (1) (2023) http://dx.doi.org/10.59720/22-059.

33. A. Tabbakh, S.S. Barpanda, Evaluation of machine learning modelsfor plant disease classification using modified GLCM and wavelet basedstatistical fea-tures, Trait. Du Signal 39 (6) (2022) 1893–1905, http://dx.doi.org/10.18280/ ts.390602.32.

34. A. Tabbakh, S.S. Barpanda, A deep features extraction model based on thetransfer learning model and vision transformer TLMViT for plant disease classification, Digit. Object Identifier, IEEE ACCESS 11 (2023) http://dx.doi.org/10.1109/ ACCESS.2023.3273317.

35. 35 S. Acharya, T. Kar, P. Patra, U. C Samal, Performance comparison between SVM and LS-SVM for rice leaf disase detection, EAI Endorsed Trans. Scalable Inf. Syst. 10 (6) (2023) http://dx.doi.org/10.4108/eetsis.3940.